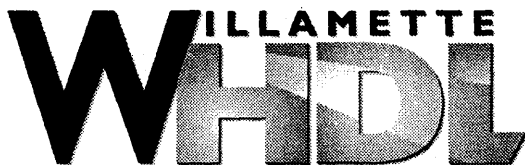


SystemVerilog

Quick Reference Guide
(Including AVM v3.0)



WILLAMETTE HDL, INC
14314 SW Allen Blvd., Suite 625
Beaverton, OR 97005
(503) 590-8499
www.whdl.com

Editors: Kurt Schwartz
Tim Corcoran
Mike Baird

V3.6: January 2008

Copyright © 2008 by Willamette HDL, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written consent of Willamette HDL, Inc.

Printed in the United States of America

SECTION 1: DATA TYPES	1
1.1 WHITE SPACE	1
1.2 COMMENTS	1
1.3 IDENTIFIERS	1
1.4 LOGIC LEVELS & STRENGTHS	1
1.5 INTEGER LITERALS	2
1.6 REAL NUMBERS	2
1.7 NET DATA TYPES (AS VERILOG)	2
1.8 SIGNED AND UNSIGNED	3
1.9 PARAMETER DATA TYPES	3
1.10 BASIC REGISTER DATA TYPES	3
1.11 INTEGER DATA TYPES	4
1.12 ENUMERATED TYPES (SV)	4
1.12.1 Enumeration methods	4
1.13 USER DEFINED TYPES (SV)	5
1.14 STATIC CASTING (SV)	5
1.15 DYNAMIC CASTING (SV)	5
1.16 ALIASING (SV)	5
1.17 STRING TYPE (SV)	6
1.17.1 String Methods	6
1.17.2 String Operators	6
SECTION 2: ARRAYS AND STRUCTURES	7
2.1 ARRAY DECLARATIONS	7
2.1.1 Packed array	7
2.1.2 Unpacked array	7
2.1.3 Array index referencing	7
2.1.4 Unpacked array literals	7
2.1.5 Variable Vector part-select	7
2.2 DYNAMIC ARRAYS (SV)	8
2.2.1 new operator	8
2.2.2 delete method	8
2.2.3 size method	8
2.3 ASSOCIATIVE ARRAYS (SV)	8
2.3.1 Associative array literals	8
2.3.2 Associative Array Methods	9
2.4 ARRAY QUERYING FUNCTIONS	9
2.5 QUEUES (SV)	10
2.5.1 Queue methods	10
2.6 ARRAY MANIPULATION METHODS (SV)	11
2.7 STRUCTURES (SV)	13
2.8 UNIONS (SV)	13

SECTION 3: PROGRAM CONTROL	14
3.1 OPERATORS	14
3.2 PROGRAMMING STATEMENTS	15
3.2.1 <i>if statement</i>	15
3.2.2 <i>case statement</i>	15
3.2.3 <i>Pattern matching case (SV)</i>	16
3.2.4 <i>forever loop</i>	16
3.2.5 <i>while loop</i>	16
3.2.6 <i>repeat loop</i>	16
3.2.7 <i>do loop (SV)</i>	16
3.2.8 <i>for Loop</i>	17
3.2.9 <i>foreach Loop (SV)</i>	17
3.2.10 <i>break and continue (SV)</i>	17
3.2.11 <i>return statement (SV)</i>	17
3.3 FINAL BLOCK (SV)	17
3.4 SYSTEM TASKS AND FUNCTIONS	18
3.4.1 <i>Text formatting codes</i>	21
3.4.2 <i>Preprocessor directives</i>	21
SECTION 4: CONTINUOUS ASSIGNMENTS	24
4.1 EXAMPLE CONTINUOUS ASSIGNMENT OF VARIABLES (SV)	24
SECTION 5: TASKS AND FUNCTIONS	25
5.1 TASK DECLARATION	25
5.2 FUNCTION DECLARATION	25
5.3 ARGUMENT LISTS	26
5.4 PASSING ARGUMENTS BY NAME	26
5.5 STATIC AND AUTOMATIC VARIABLES	26
SECTION 6: PROCESSES	27
6.1 PROCEDURAL BLOCKS	27
6.1.1 <i>Procedural Timing Controls</i>	27
6.1.2 <i>Procedural Sensitivity lists</i>	28
6.1.3 <i>Sequential Block</i>	28
6.1.4 <i>Procedural Assignments</i>	29
6.1.5 <i>Blocking vs Non-blocking assignmnet: Rules of Thumb</i>	29
6.2 DYNAMIC PROCESSES	30
6.2.1 <i>fork join</i>	30
6.2.2 <i>fork join_any (SV)</i>	30
6.2.3 <i>fork join_none (SV)</i>	30
6.2.4 <i>wait fork (SV)</i>	31
6.2.5 <i>disable fork (SV)</i>	31
6.2.6 <i>Dynamic Process Methods (SV)</i>	31
6.3 INTER-PROCESS SYNCHRONIZATION AND COMMUNICATION	31
6.3.1 <i>Named Events</i>	31
6.3.2 <i>Triggering events</i>	32
6.3.3 <i>Persistent trigger (SV)</i>	32

6.3.4 Event sequencing (SV)	32
6.3.5 Semaphores (SV)	32
6.3.6 Semaphore Methods.....	33
6.3.7 Mailboxes (SV).....	33
6.3.8 Mailbox Methods.....	33
SECTION 7: HIERARCHY	34
7.1 MODULES	34
7.2 MODULE PORT DECLARATIONS	34
7.3 MODULE INSTANCES.....	35
7.3.1 Example - Module Instantiation.....	36
7.3.2 Example - Parameterized Module Instantiation.....	36
7.3.3 Implicit Port Connections (SV)	37
7.3.4 .name Syntax.....	37
7.3.5 .* Syntax.....	37
7.4 PRIMITIVES.....	37
7.4.1 User defined Primitives (UDPs)	37
7.5 SPECIFY BLOCKS.....	38
7.6 PACKAGES (SV).....	38
7.7 REFERENCING PACKAGE ITEMS	39
7.7.1 Fully qualified	39
7.7.2 Via importing.....	39
7.8 EXPORTING PACKAGE ITEMS	39
7.8.1 Example of package export	39
7.9 TIME UNIT AND PRECISION (SV)	40
7.10 PROGRAM BLOCKS (PB) (SV)	40
7.11 CLOCKING BLOCKS (CB) (SV)	41
7.11.1 Example clocking block.....	41
7.11.2 Example using a Clocking Block	42
7.12 INTERFACES (SV)	43
7.12.1 Interface Declaration.....	43
7.12.2 Modport Declaration	43
7.13 INSTANTIATING INTERFACES.....	44
7.13.1 Example 1 (modport list in module header)	44
7.13.2 Example 2 (modport list in module instantiation)	44
SECTION 8: GENERATE, CONFIG & LIBRARY MAP FILES.....	45
8.1 GENERATE BLOCKS	45
8.2 LIBRARY MAPPING FILES	48
8.3 CONFIGURATIONS	49
8.4 EXAMPLE OF LIBRARY & CONFIG.....	49

SECTION 9: CLASSES	50
9.1 CLASS DECLARATION	50
9.2 CLASS CONSTRUCTORS	50
9.3 OBJECT INSTANTIATION	51
9.4 CONSTRUCTOR CHAINING	52
9.5 COPYING OBJECTS	53
9.6 CONTROLLING ACCESS TO MEMBERS	53
9.7 VIRTUAL METHODS	53
9.8 VIRTUAL (ABSTRACT) CLASSES	54
9.8.1 <i>Pure Virtual Methods</i>	54
9.9 DEFINING METHODS OUTSIDE CLASS DECLARATION	54
9.10 PARAMETERIZED CLASSES	55
9.11 VIRTUAL INTERFACE	56
SECTION 10: RANDOMIZATION & CONSTRAINTS (SV)	57
10.1 RAND AND RANDC	57
10.2 ENABLING AND DISABLING RANDOMIZATION	57
10.3 CONSTRAINT BLOCK	58
10.4 CONSTRAINT BLOCK CONSTRUCTS	58
10.4.1 <i>foreach</i>	58
10.4.2 <i>implication (->, if else)</i>	58
10.4.3 <i>dist</i>	58
10.4.4 <i>Solution order</i>	59
10.5 IN-LINE CONSTRAINTS	59
10.6 ENABLING AND DISABLING CONSTRAINTS	60
10.7 RANDOMIZING NON-CLASS VARIABLES	60
10.8 PRE AND POST RANDOMIZATION	60
10.9 MANUAL SEEDING	60
10.10 RANDOM WEIGHTED CASE	61
10.11 RANDOM SEQUENCE GENERATION	61
10.12 RANDOM NUMBER SYSTEM FUNCTIONS	61
10.13 RANDOM DISTRIBUTION FUNCTIONS	61
SECTION 11: FUNCTIONAL COVERAGE (SV)	62
11.1 COVERGROUP DECLARATION	62
11.2 COVERPOINT DECLARATION	62
11.2.1 <i>Value bins</i>	62
11.2.2 <i>Transition bins</i>	62
11.3 CROSSPOINT COVERAGE	63
11.4 COVER OPTIONS	63
11.5 COVER TYPE OPTIONS	64
11.6 COVER FUNCTIONS	64
11.7 COVERAGE SYSTEM FUNCTIONS	64

SECTION 12: SYSTEMVERILOG ASSERTIONS (SV)	65
12.1 IMMEDIATE ASSERTIONS	65
12.2 SEVERITY SYSTEM TASKS	65
12.3 CONCURRENT ASSERTIONS	65
12.4 PROPERTY DECLARATIONS	65
12.5 ASSERTION DIRECTIVES	66
12.6 EXPECT STATEMENT	66
12.7 PROPERTY EXPRESSIONS	67
12.8 SEQUENCE BLOCKS	67
12.9 SEQUENCE EXPRESSIONS	67
12.10 SEQUENCE ENDPOINT	67
12.11 SEQUENCE MATCHPOINT	67
12.12 SEQUENCE OPERATORS	68
12.13 SEQUENCES IN PROCEDURAL CODE	68
12.14 DATA-USE WITHIN A SEQUENCE OR PROPERTY	69
12.15 SUBROUTINE CALLS ON MATCH OF A SEQUENCE	69
12.16 ASSERTION SYSTEM FUNCTIONS	70
12.17 VALUE CHANGE FUNCTIONS	70
12.18 BIND DIRECTIVE	71
12.19 ASSERTION CONTROL	71
SECTION 13: AVM QUICK REFERENCE	72
13.1 NAMED COMPONENT	72
13.2 THREADED COMPONENT	73
13.3 ENVIRONMENT	73
13.4 TRANSACTION	74
13.5 POLICIES	74
13.6 UTILITIES	75
13.7 TLM CHANNELS	76
13.8 TLM INTERFACES	77
13.9 REPORTING	79
13.10 PORTS AND EXPORTS	80
SECTION 14: SYSTEMVERILOG KEYWORD INDEX	82

Introduction

This guide is a convenient reference for the SystemVerilog language. For completeness, it covers much of the Verilog 1364 language as well. It is intended for users writing models and testbenches. The topics have been simplified and optimized toward those aspects of the language that are most commonly used.

The final section is a Quick Reference to the Mentor Graphics Advanced Verification Methodology AVM v3.0.

For a complete, authoritative language definition, the user should refer to the official IEEE specifications published by the IEEE at <http://www.ieee.org>

SystemVerilog Standard 1800-2005 Language Reference Manual
Verilog Standard 1364-2001 Language Reference Manual

Every topic follows a similar format:

Topic

Syntax descriptions are in a shaded box

Following the syntax description, notes about the syntax and semantics of the topic are detailed.

The syntax descriptions are not strictly formal, as in BNF, but instead follow a somewhat simplified convention:

- Keywords and required symbols are in **bold** format.
- Items appearing between non-bold brackets [] are optional.
- An ellipsis ... means that the previous clause can be repeated indefinitely.
- *Italicized* text is explanatory in nature and not a formal part of the syntax.
- (*SV*) in headings indicates a SystemVerilog specific construct.

Section 1: Data Types

1.1 White Space

[System]Verilog treats *space*, *tab*, *newline* or *carriage return*, *form* and *line-feed* characters as white space and ignore them except as needed to separate lexical keywords from identifiers and other text.

1.2 Comments

```
// start of single line comment (terminate by newline)

/* start of a multi-line comment
   next line ( */ ) terminates this multi-line comment
*/
```

1.3 Identifiers

- Names given to [System]Verilog modules, nets, variables, instances etc.
- Must begin with an alphabetical (upper or lower case) or _ (underscore) and terminates with white space
- Subsequent characters may include alpha-numerics, \$ or _
- Other characters may be used if identifier begins with \

1.4 Logic Levels & strengths

[System]Verilog has 7 logic strengths as well as high impedance (no strength).

Strength	Name	Keywords
7	Supply Drive	supply0, supply1
6	Strong Drive	strong0, strong1
5	Pull Drive	pull0, pull1
4	Large Capacitive	large
3	Weak Drive	weak0, weak1
2	Medium Capacitive	medium
1	Weak Capacitive	small
0	High Impedance	highz0, highz1

1.5 Integer Literals

Base	Syntax	Legal Values
hexadecimal	size 'h value	x, X, z, Z, ?, _ 0-9, a-f, A-F
decimal	size 'd value -or- value	_, 0-9
	size 'sd value	
octal	size 'o value	x, X, z, Z, ?, _, 0-7
	size 'so value	
binary	size 'b value	x, X, z, Z, ?, _, 0, 1
	size 'sb value	

Notes:

- Size is a number of binary bits, value is a literal in the appropriate base
- The default base is decimal
- The underscore (_) is ignored
- ?, z and Z are synonymous representations of high impedance
- If **size** is **fewer bits** than **value**, upper bits of **value** are truncated
- If **size** is **more bits** than **value**, **value** is extended to the left according to the following rules:
 - If leftmost bit of value is 1 or 0, extending bits are 0
 - If leftmost bit of value is X, extending bits are X
 - If leftmost bit of value is Z, extending bits are Z
- Signed literals (e.g. 5'sb10101) imply 2's complement values.

1.6 Real numbers

Reals are expressed in decimal (0-9) or scientific notation (e.g. 3e6)

1.7 Net Data Types (as Verilog)

Type	Notes
wire, tri	Standard Interconnect type
wor, wand	Wired logic resolution type
tri0, tri1	Pull up/down when tristated
triereg	Store last value when tristated (like CMOS)
supply0, supply1	Constant supply

Notes:

- The table above shows the Verilog-95 interpretation of a net.
For backward compatibility this is fully supported in SV.
- SV also allows any 4-state data type to be used to declare a net

1.8 Signed and unsigned

Since Verilog 2001 net, register and port declarations may include the word **signed**. For nets and ports this allows the entire datapath thru a model to correctly reflect that signed values are being passed.

In the case of reg types it also provides for any-width signed integer arithmetic. In Verilog 1995 only 32 bit signed arithmetic was allowed because the only signed type was integer (always 32 bits).

The keyword **unsigned** (*SV*) has the opposite effect, converting datatypes that are signed by default into unsigned types. Both words affect the meaning of certain operators

Example: Signed adder module

```
module adder( output reg signed [7:0] sum,
              input wire signed [7:0] ina, inb );
    ...
endmodule
```

1.9 Parameter Data Types

Type	Notes
parameter	Elaboration-time constant
parameter type	Elab-time constant type specifier
localparam	Same as parameter except not modified by defparam
specparam	Specify block constant
const	Run-time constant qualifier

Notes:

- Parameter type is a SystemVerilog feature permitting parameterization with respect to type as well as numeric constants. Example in Section: 7.3.2

1.10 Basic Register Data Types

Type	Notes
time	64-bit integer, defaults to seconds
real	From Verilog, like C double, 64 bits
shortreal	From C float, 32 bits
string	Variable-sized array of char
void	Non-existent data, used for functions

1.11 Integer Data Types

Type	Notes	States	Default Val
Signed			
shortint	16-bit integer	2	0
int	32-bit integer	2	0
longint	64-bit integer	2	0
byte	8-bit integer (usually same as char)	2	0
integer	32-bit from Verilog (sized: 0, 1, X, Z)	4	X
Unsigned			
bit	0 or 1	2	0
reg	From Verilog (unsized: 0, 1, X, Z)	4	X
logic	Like reg, but with different usage rules	4	X

Notes:

- Only *integer* and *reg* are defined in the Verilog HDL.

1.12 Enumerated Types (SV)

```
enum [enum_base_type] { name_decl, name_decl } varname
```

1.12.1 Enumeration methods

function enum first ();	Returns value of the first enum member
function enum last ();	Returns value of the last enum member
function enum next (int unsigned N=1);	Returns the Nth next enum value starting from the current value
function enum prev (int unsigned N=1);	Returns the Nth previous value starting from the current value
function int num ();	Returns the number of elements in the given enumeration
function string name ();	Returns the string representation of the current value

Notes:

- *enum_base_type* defaults to int.
- enum name declarations can be assigned values and will auto-increment from there.
- Default assigned values start at 0.

1.13 User Defined Types (SV)

```
typedef [base_type] type_identifier
```

1.14 Static Casting (SV)

```
type' ( value )
```

Notes:

- Convert to a specific type with the cast (forward-tick) operator. Checking is performed at compile time, hence the name static

1.15 Dynamic Casting (SV)

Prototype:

```
function int $cast( singular_dest_var, source_expr );  
  
task $cast( singular_dest_var, source_expr );
```

Notes:

- \$cast may be called as a function or task. As a function it returns 1 if cast succeeds, 0 if it fails. As a task, failure triggers a runtime error

1.16 Aliasing (SV)

The alias statement acts like a bidirectional short-circuit between nets.

```
alias a_net = b_net
```

Notes:

- Variables and hierarchical references are not allowed.
- Net types must be the same.
- A signal may not be aliased to itself.
- Aliasing is performed at elaboration time and may not be undone.

```
module byte_swap(inout tri[31:0] A, inout tri[31:0] B);  
  
alias {A[7:0],A[15:8],A[23:16],A[31:24]} = B;  
  
endmodule
```

Notes:

- Example of a bi-directional byte-swap between A and B

1.17 String Type (SV)

string varname

Notes:

- A variable size array of characters indexed from 0(LHS) to N-1(RHS)

1.17.1 String Methods

function integer atobin (); function integer atohex (); function integer atoi (); function integer atooct ();	Returns the integer corresponding to the ASCII character provided.
function real atoreal ();	Converts a string representation of a real to a real.
function void bintoa (integer i);	Stores the ASCII binary representation of i into the string.
function int compare (string s);	Compares the string to s, as in the ANSI C function strcmp.
function byte getc (int n);	Returns the ASCII code of the n-th character.
function void hextoa (integer i);	Stores the ASCII hexadecimal representation of i into the string.
function int icompare (string s);	Like compare, but case insensitive.
function void itoa (integer i);	Stores the ASCII decimal representation of i into the string.
function int len ();	Returns the string length.
function void octtoa (integer i);	Stores the ASCII octal representation of i into the string.
function void putc (int i, byte c);	Replaces the ith character of the string with the given integral value.
function void realtoa (real r);	Stores the ASCII real representation of r into the string.
function string substr (int i, int j);	Returns a new string that is a substring formed by the characters in position i through j of the string.
function string tolower ();	Returns a new string with characters of the string converted to lowercase.
function string toupper ();	Returns a new string with characters of the string converted to uppercase.

1.17.2 String Operators

Supports comparison (==, !=, <, <=, >, >=), concatenation ({ }), and replication ({n{ } }).

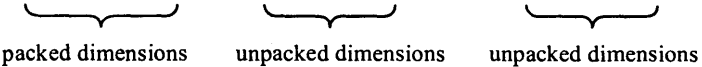
Notes:

Comparisons are made alphabetically.

Section 2: Arrays and Structures

2.1 Array Declarations

```
datatype [a:b][c:d]... var1[s:t][u:v]... , var2[w:x][y:z]...
```



As an alternative to a range [n:m], unpacked dimensions may be specified as a single positive number [size] which will be taken to mean [0: size -1]

2.1.1 Packed array

A packed array is a contiguous set of bits that can be subdivided logically into separate addressable subfields. Packed arrays can only be made from bit, reg, logic, or other packed arrays.

2.1.2 Unpacked array

An unpacked array is a set of variables that can be logically addressed by indexed fields, but the storage for the set of variables is not guaranteed to be contiguous. An unpacked array variable can be any SystemVerilog type.

2.1.3 Array index referencing

When referencing an index of an array, the unpacked dimensions are specified first, followed by the packed dimensions. Where multiple dimensions exist they are addressed in order left-to-right.

2.1.4 Unpacked array literals

An unpacked array may be assigned by means of the replicate operator {{}}.

```
int j [1:5][1:2] = '{default: 5}; // All elements "5"

int k [3][4] = '{ {1,2,3,4}, {5,6,7,8}, {9,10,11,12} };
// think of array as 3 groups of 4 values
```

2.1.5 Variable Vector part-select

```
vector[start_bit +: width]
vector[start_bit -: width]
```

Notes:

- +: indicates that part-select increments from *start_bit* index
- : indicates that part-select decrements from *start_bit* index
- width* must be an integer literal

2.2 Dynamic Arrays (SV)

Unpacked array dimensions can be left unspecified in the declaration, and allocated dynamically at runtime with the **new** operator.

```
bit [3:0] nibble[]; // Dynamic array of 4-bit vectors
initial
```

```
    nibble = new[100]; // Allocate and initialize 100 elements
```

```
integer mem[] = new[5]; // combined declaration/allocation/initialization
```

2.2.1 new operator

```
arrayvar = new [size];
arrayvar = new [size] (varname);
```

Notes:

- `new[]` may be used to allocate memory for a new array or to resize an existing array.
- `varname` is optional but if it is present and the array already exists, the appropriate range of content from `varname` are copied to `arrayvar`.

2.2.2 delete method

```
arrayvar.delete()
```

Notes:

- Delete array contents and zero-size it.

2.2.3 size method

```
arrayvar.size()
```

Notes:

- Returns the current size of a dynamic array or returns zero if the array has not been created.

2.3 Associative Arrays (SV)

```
arraytype varname [index_type];
```

Notes:

- The index type does not have to be an integer.
- If specified as `*` the index can be any integral value.

2.3.1 Associative array literals

```
{ idxval:elemval, ..., default:elemval }
```


2.3.2 Associative Array Methods

<code>function int num();</code>	Returns number of elements in array.
<code>function void delete ([input index]);</code>	Removes specified element from array
<code>function int exists ([input index]);</code>	Returns true if the element associated with the index value exists in the array.
<code>function int first(ref index);</code>	Assigns to the given index variable the value of the first (smallest) index in the array.
<code>function int last(ref index);</code>	Assigns to the given index variable the value of the last (largest) index in the array.
<code>function int next(ref index);</code>	Finds the entry whose index is greater than the specified index. If there is a next entry, the index variable is assigned the index of the next entry, and the function returns 1. Otherwise, the index is unchanged, and the function returns 0.
<code>function int prev(ref index);</code>	Finds the entry whose index is smaller than the given index. If there is a previous entry, the index variable is assigned the index of the previous entry, and the function returns 1. Otherwise, the index is unchanged, and the function returns 0.

Notes:

- The argument that is passed to any of the four associative array traversal methods `first()`, `last()`, `next()`, and `prev()` shall be assignment compatible with the index type of the array.

2.4 Array Querying Functions

<code>\$left</code>	Returns the left bound (MSB) of the dimension.
<code>\$right</code>	Returns the right bound (LSB) of the dimension.
<code>\$low</code>	Returns the minimum of <code>\$left</code> and <code>\$right</code> .
<code>\$high</code>	Returns the maximum of <code>\$left</code> and <code>\$right</code> .
<code>\$increment</code>	Returns 1 if <code>\$left</code> $\geq$ <code>\$right</code> and -1 if <code>\$left</code> < <code>\$right</code> .
<code>\$size</code>	Returns the number of elements in the dimension.
<code>\$dimensions</code>	Returns one of: - The total number of dimensions in the array. - 1 for string and other non-array vector types - 0 for any other type.
<code>\$unpacked_dimensions</code>	Returns the total number of unpacked dimensions for an array, or 0 for any other data type.

Notes:

- All returns are integer type

2.5 Queues (SV)

```
element_t varname[$];
```

Notes:

- The \$ symbol between [] is used to declare a queue.
- Each element in a queue is identified by an ordinal number that represents its position within the queue, with 0 representing the first, and \$ representing the last.

2.5.1 Queue methods

function int size ();	Returns number of items in the queue. Returns 0 if the queue is empty
function void insert (input int index, input element_t item);	Inserts the given item at the specified index location.
function void delete (int index);	Deletes the item at the specified index location.
function element_t pop_front ();	Removes and returns the first element of the queue.
function element_t pop_back ();	Removes and returns the last element of the queue.
function void push_front (input element_t item);	Inserts the given item at the front of the queue.
function void push_back (input element_t item);	Appends the given item at the end of the queue.

Notes:

- element_t is the type of the queue
- To zero-size a queue assign with an empty literal (e.g. q = '{};)

Example using queue in a behavioral FIFO

```
module q_fifo( input logic[7:0] d_in,
               output bit mt, full, logic[7:0] d_out );
    logic [7:0] q [$];

    always @ (test_q_fifo.write_fifo)
        if(full) $display ("tried to write a full fifo");
        else begin
            q.push_front(d_in);
            if(q.size() > 7) // MAX fifo depth is 7
                full = 1; mt = 0;
            end

    always @ (test_q_fifo.read_fifo)
        if(mt) $display("tried to read an empty fifo");
        else begin
            d_out <= q.pop_back();
            full = 0;
            if(q.size() == 0) mt = 1;
            end
    endmodule
```

2.6 Array Manipulation Methods (SV)

`arrayname.methodname(args) with (expression)`

Locator Methods (with-clause mandatory) – returns a queue	
<code>find()</code>	Returns a queue of all elements satisfying the given expression.
<code>find_index()</code>	Returns a queue of index values of all elements satisfying the given expression.
<code>find_first()</code>	Returns the first element that satisfies expression.
<code>find_first_index()</code>	Returns the index of the first element that satisfies the expression.
<code>find_last()</code>	Returns the last element that satisfies the expression.
<code>find_last_index()</code>	Returns the index of the last element that satisfies the expression.
Locator Methods (with-clause is optional) – returns a queue	
<code>min()</code>	Returns the element with the minimum value or whose with -expression evaluates to a minimum.
<code>max()</code>	Returns the element with the maximum value or whose with -expression evaluates to a maximum.
<code>unique()</code>	Returns all elements with unique values.
<code>unique_index()</code>	Returns the indexes of all elements with unique values.
Ordering Methods – returns a void type	
<code>reverse()</code>	Reverses the order of the elements in the array. The with -clause is illegal for this method.
<code>sort()</code>	Sorts the elements in ascending order.
<code>rsort()</code>	Sorts the elements in descending order.
<code>shuffle()</code>	Randomizes the order of elements. The with -clause is illegal for this method.
Reduction Methods – see note below for return type	
<code>sum()</code>	Returns the sum of the elements, or the sum of the with -expression values.
<code>product()</code>	Returns the product of the elements, or the product of the with -expression values.
<code>and()</code>	Returns the bitwise and of the elements, or of the with -expression values.
<code>or()</code>	Returns the bitwise or of the elements, or of the with -expression values.
<code>xor()</code>	Returns the bitwise xor of the elements, or of the with -expression values.
Index Query – see note below for return type	
<code>index(dim)</code>	Returns an int index value

Notes:

- Locator methods return a queue of the same as the array on which they operate except for associative arrays which return a queue of the same type as the associative index type.
- Ordering methods return void type
- Reduction methods return the same as the array type or if specified, the with-expression type.
- For associative arrays the *index()* function returns the same type as the array, for all other arrays it returns int.

2.7 Structures (SV)

```
struct [packed [signing]] { member, member } sname ...
```

Notes:

- 2 styles: **packed** (subdividable vector), **unpacked** (implementation dependant)
- Default is unpacked. Add **packed** qualifer to override.
- Unpacked structures may be any SV type including arrays & other structs/unions.
- Packed structures may contain only integer types, no unpacked types, including structs/arrays.
- Members may be referenced by syntax: sname.member
- May be typedef'd
- May be passed to/from functions or tasks.
- May be assigned as a whole

```
typedef struct {  
    logic [7:0] a;  
    int b;  
} my_struct;  
  
my_struct s1 ;  
  
initial begin  
    #10 s1 = '{5, 6};           // assign by position  
    #10 s1 = '{b:5, a:6};       // assign by name  
    #10 s1 = '{default: 7};     // all members assigned 7  
    #10 s1 = '{int:9, default:1}; // assign by type,  
                                // others default  
end
```

2.8 Unions (SV)

```
union [tagged [packed [signing]]] {member, ..} uname ...
```

Notes:

- Default is unpacked (similar to C++), not much use in H/W.
- Many similar characteristics to structures (above)
- Packed form are the most useful in H/W.
 - All members are same size and occupy same memory space.
 - May be written via one member and read by another.
 - Only integer datatype members allowed.
 - Unpacked arrays/structs not allowed.
- Tagged unions are strongly type-checked. Updates must meet consistency tests to prevent type misuse. See LRM for more details.

Section 3: Program Control

3.1 Operators

Operator	Data Type
Unary Logical Operators	
~ & ~& ~ ^ ~^ ^~	Integral
!	Integral, real, shortreal
Unary Arithmetic Operators	
+ -	Integral, real, shortreal
Binary Arithmetic Operators	
+ - * / **	Integral, real, shortreal
Binary Logical Operators	
% & ^ ^~ ~^ >> << >>> <<<	Integral
! &&	Integral, real, shortreal
Relational Operators	
< <= > >=	Integral, real, shortreal
Case Equality Operators	
=== !=	Any <i>except</i> real and shortreal
Logical equality Operators	
== !=	Any
Wildcard Equality Operators	
==? !=?	Integral
Increment and Decrement Operators (SV)	
++ --	Integral, real, shortreal
Set Membership Operator (SV)	
inside	Singular for the left operand
Concatenation and Replication Operators	
{ } { }	Integral
Distribution Operator (SV)	
dist	Integral
Conditional Expression Operator	
?:	Any
C-Style Assignment Operators (SV)	
+= -= *= /=	Integral, real, shortreal
%= &= = ^= <<= >>= <<<= >>>=	Integral

3.2 Programming Statements

3.2.1 if statement

```
[unique_or_priority] if (expr) stmt1 [ else stmt2 ] ;
```

Notes:

- While the else clause is optional, this is generally to be avoided in Synthesis as it is likely to infer latches in combinatorial always blocks.
- In SystemVerilog, an if statement can be preceded by the keywords unique or priority.
 - unique asserts that all the expressions in an if .. else if ... chain are mutually exclusive.
 - priority asserts that the expressions in the if ... else if ... chain should be evaluated in the order listed.

3.2.2 case statement

```
[unique_or_priority] case (expr)  
  case_item1 : stmt1 ;  
  ...  
  [ default : stmt2 ; ]  
endcase
```

Notes:

- The expr and case_item are compared using the identity operator (===).
- If no match is found, the (optional) default branch is executed.
- While the default clause is optional, this is generally to be avoided in synthesis as it is likely to infer latches in combinatorial always blocks.
- In SystemVerilog, a case statement can be preceded by the keywords unique or priority.
- A priority case acts on the first match only.
- A unique case asserts that the items are mutually exclusive.
- Two variants of the case statement, casex and casez, allow for wildcarding of the expr or case_items.
 - In the casez variant, a ? or Z value is a wildcard.
 - In the casex variant, a ?, Z, or X value is a wildcard.

3.2.3 Pattern matching case (SV)

```
[unique_or_priority] case (expr) matches
  pattern [ &&& filter_expr ] : stmt;
...
endcase

pattern can be one of:
.*
.var_id
.constant_expr
tagged member_id [pattern]
`{ pattern [, pattern, ...] }
`{ member_id:pattern [, member_id:pattern, ...] }
```

Notes:

- The .* wildcard pattern matches anything
- Pattern matching also applies to **if** statements and conditional expressions
(sel ? a : b;)

3.2.4 forever loop

```
forever statement
```

Notes:

- Executes statement(s) in an infinite loop.

3.2.5 while loop

```
while ( expression ) statement
```

Notes:

- Evaluates the expression. If the expression is non-zero, executes statement(s), then repeats.

3.2.6 repeat loop

```
repeat ( expression ) statement
```

Notes:

- Repeats the statement(s) a number of times according to the result of the expression.

3.2.7 do loop (SV)

```
do statement while expression
```

Notes:

- A do loop is like a while loop, but the conditional loop expression is evaluated after the loop iteration rather than before.

3.2.8 for Loop

```
for ( declare init_assign [ ,declare init_assign ] ;  
limit_expression ; loop_update [, loop_update ] )
```

Notes:

- Loop exits if the result of `limit_expression` is zero.
- SystemVerilog allows the loop iterator to be declared locally within the loop and permits multiple loop iterators and `loop_update` assignments.

3.2.9 foreach Loop (SV)

```
foreach ( arrayname [ loopvar [, loopvar ...]] ) statement
```

Notes:

- Iterates over the elements of an array. `loopvar` corresponds to a dimension of the array.

3.2.10 break and continue (SV)

SystemVerilog adds the `break` and `continue` statements from C.

break	Jumps to the statement immediately following the current loop.
continue	Immediately starts a new iteration of the current loop.

3.2.11 return statement (SV)

```
return  
return [ expr ]  
return [ ( expr ) ]
```

Notes:

- Exits from a task or function. The optional return expression is only valid in non-void functions.

3.3 final block (SV)

```
final statement_block
```

Notes:

- The final block is like an initial block, except that it runs at the end of simulation.
- A final block must execute without delays.

3.4 System Tasks and Functions

These utility tasks and functions provide some broadly useful capabilities such as message printing, file i/o, casting and time specification.

```
$display("format'd_message", signal, signal, ... );  
$displayo("format'd_message", signal, signal, ... );  
$displayh("format'd_message", signal, signal, ... );  
$displayb("format'd_message", signal, signal, ... );
```

- Prints the formatted message to standard out.
- Inserts a newline at the end of every message.
- There may be more than one formatted message argument.

```
$write("formatted_message", signal, signal, ... );  
$writeo("formatted_message", signal, signal, ... );  
$writeh("formatted_message", signal, signal, ... );  
$writeb("formatted_message", signal, signal, ... );
```

- Like **\$display**, except that no newline is added.

```
$monitor("format'd_message ", signal, signal, ... );  
$monitro("formt'd_message ", signal, signal, ... );  
$monitorh("formt'd_message ", signal, signal, ... );  
$monitorb("formt'd_message ", signal, signal, ... );
```

- Continuously monitors the signals listed, and prints the formatted message whenever any of the signals change. Notice default base (o,h,b) is specifiable.

```
$strobe("formatted_message", signal, signal, ... );  
$strobo("formatted_message", signal, signal, ... );  
$strobeh("formatted_message", signal, signal, ... );  
$strobeb("formatted_message", signal, signal, ... );
```

- Like **\$display**, except printing is delayed until the end of the current time step.

```
mcd = $fopen("file_name");
```

- Opens a file for writing, and returns the 32-bit integer multi-channel-descriptor (mcd) to the file.

```
$fclose(mcd);
```

- Closes a file previously opened by **\$fopen**.

```
$fmonitor(mcd, "formatted_message", signal, ... );  
$fdisplay(mcd, "formatted_message", signal, ... );  
$fwrite (mcd, "formatted_message", signal, ... );  
$fstrobe (mcd, "formatted_message", signal, ... );
```

- Write to files previously opened with **\$fopen**.

```
$swrite(string, "formatted_message", signal, ... );  
$swriteo(string, "formatted_message", signal, ... );  
$swriteh(string, "formatted_message", signal, ... );  
$swriteb(string, "formatted_message", signal, ... );
```

- Like **\$fwrite**, except that first argument is of type string.
- Sends formatted output to target string instead of a file

```
$sformat(string, "format_string", signal, ... );
```

- Like **\$swrite** except that second argument (and only this argument) is interpreted as a format string.
- The format string argument can be a string literal ("**result = %0h**") or a string type variable whose content is interpreted as a format string.
- Often used as a specialized **\$swrite** to build strings from other strings.

```
$time
```

- Returns current simulation time as a 64-bit integer.

```
$stime
```

- Returns lower 32-bits of simulation time as an integer.

```
$realtime
```

- Returns current simulation time as a real.

```
$timeformat(unit, precision, "suffix", min_width);
```

- Sets the format used by the **%t** text format specifier.
unit - base to display time in, from 0 to -15
 - 0 = 1sec
 - 1 = 100ms
 - 9 = 1ns etc.
precision - number of decimal points to display.
suffix - string appended to the time, such as " ns".
min_width - minimum # of characters displayed.

```
$sprinttimescale( hierarchical_name );
```

- Prints the time scale of the specified module, or the scope from which it is called if no module is specified.

\$random(seed) ;

- Returns a random 32-bit signed integer.

\$readmemb("file_name", register_array, start, end) ;
\$readmemh("file_name", register_array, start, end) ;

- Load register_array from an ASCII formatted file. The file contains values in binary (\$readmemb) or hex (\$readmemh). The start and end arguments (array addresses) are optional.

\$finish[(n)] ;

- Finishes simulation and exits to the operating system.

\$stop[(n)] ;

- Halts a simulation and enters interactive debug mode.

\$exit ;

- Waits for all program blocks to complete, then makes an implicit call to \$finish.

Notes:

\$finish and **\$stop** system tasks accept an optional argument *n* which controls how much diagnostic information is printed.

Diagnostic argument to **\$finish** and **\$stop**

Argument value	Diagnostic information printed
0	Prints nothing
1 (default)	Prints simulation time and location
2	Prints simulation time and location <u>plus</u> statistics about memory/cpu utilization

3.4.1 Text formatting codes

The following formatting codes are used in declaring the “formatted_message” field above.

%b	binary	%m	hierarchical instance name
%o	octal		
%d	decimal		
%h	hexadecimal		
%e	real exponential	\t , \n	print tab, newline
%f	real floating decimal	\" , \\	print double-quote, backslash
%t	formatted time	%	print percent sign
%s	text string	\v , \f	print vertical tab, form feed
%z	4-state var display	\a	ring console bell
		\x02	hex number
Zero (e.g. %0h) indicates the value should be printed with minimum width. %e, %f may define desired width on either side of decimal point (e.g. %3.2f)			

3.4.2 Preprocessor directives

Also known as compiler directives, preprocessor directives are not Verilog statements and do not respect the usual scoping rules of module/interface or program blocks. They also do not require semicolon at the end of a line.

<code>`timescale</code> time_unit base / precision base
Specifies unit and precision of delay specifiers. <i>time_unit</i> - basic delay unit. The time unit must be 1 10 or 100 <i>base</i> - time base for each unit, (s ms us ns ps or fs) <i>precision</i> - decimal points of precision
Example:
<code>`timescale 1 ns / 10 ps</code> <code>// Nanosecond units to 2 decimal places of accuracy.</code>
<code>`include</code> "file name"
File inclusion. Insert the text of another Verilog HDL source file exactly where the <code>`include</code> directive appears.
<code>`unconnected_drive pull1</code></b> <b><code>`unconnected_drive pull0</code> <code>`nounconnected_drive</code>
Sets which logic value is be applied to unconnected module inputs. Default is <code>`nounconnected_drive</code>, (high impedance).

<pre> `ifdef macro_name     source_code `else source_code `elsif     source_code `endif </pre>	<pre> `ifndef macro_name     source_code `else source_code `elsif     source_code `endif </pre>
Conditional compilation. Optional compile/ignore of Verilog source code, depending on whether macro_name has been defined or not.	

<pre> `define macro_name text_string `define macro_name (args) text_string (args) </pre>
Text substitution macro. Defines a text string as a macro name. - text_string will be substituted for macro_name in subsequent code. macro_name must be preceded by the grave accent mark (`) each time the macro name is used. - text_string – Single line of text, terminated by carriage return. - args - evaluated before text is substituted.
Example: <pre> `define period 10 // clock period reg clk; initial forever begin     clk = 0;     #(`period/2) clk = !clk; end </pre>

Example of Macro with arguments

<pre> bit[2:0][7:0]a[3:0]; int b = 1; byte c = 2; `define P(SIG) " SIG: %0h",SIG," "  initial     \$display(`P(a[3][2]), `P(b), `P(c)); Simulator output: a[3][2]: 0 b: 1 c: 2 </pre>
--

`uselib file = file dir = directory libext = filename extension
Allows inline specification of sourcecode libraries to be searched for missing modules. Far superior to command line -y or -v switches.
Example:
<pre>`uselib file=/hierarchical_path/rtl_lib ASIC1 U1 (port_list);    // Instantiate RTL model  `uselib dir=/hierarchical_path/gate_lib libext=.v ASIC1 U2 (port_list); // Instantiate Gate model</pre>

Notes:

- Remember, preprocessor directives take effect where they occur in the source and continue in effect for subsequent code/modules until end of source or until they are changed by another directive.
- **file** = - search in this *file* for any subsequent modules (not specifically provided on the command line)
- **dir** = - search in this *directory* for any subsequent modules (not specifically provided on the command line)
- **libext** = - use this *filename_extension* when searching **dir** above.

`reset all
Reset all compiler directives to their default (if any).
`undef macro_name
Remove the definition of a macro.

Section 4: Continuous Assignments

There are two forms of continuous assignment, explicit and implicit.

```
net_type [[ size ]] net_name;
```

// explicit cont. assign to an existing net type

```
assign [ #delay ] net_name = expression;
```

// implicit cont. assign combined with declaration of net type

```
net_type [[ size ]][ #delay ] net_name = expression;
```

Notes:

- In Verilog, continuous assignments may drive net datatypes only. They are used to describe combinatorial logic.
- SystemVerilog extends the use of continuous assignments to non-net types (i.e. variables of any type) provided the variable is driven by only one of the following:
 - A single continuous assignment
 - A single primitive or module output port
 - Some number of procedural assignments (including initialization)

These rules are required because in SystemVerilog, only the net datatype is resolved and these rules ensure resolution is not needed for variables.

- Continuous assignments must occur outside of procedural blocks and are executed concurrently with procedural blocks, primitives etc.
- net_type may be any of the net types *except* trireg.
- Optional delay is inertial and defaults to zero.

4.1 Example continuous assignment of variables (SV)

```
wire [7:0] in_a;
```

```
byte in_b;
```

```
byte var_c = 0;
```

```
struct { byte a } a_struct = sel ? in_a : in_b;
```

// Following assign is ILLEGAL because var_c already

// procedurally assigned by initializer above

```
assign var_c = in_a;
```


Section 5: Tasks and Functions

There are two types of procedural subroutine in Verilog.

Functions:

- Callable in-line, e.g. as an operand within an expression, so return value of the function is used in calculating the expression.
- Requires at least one input argument.
- Must execute in a single time-step (no timing constructs).
- Blocking assignments only (no non-blocking constructs).
- Cannot call tasks but may call functions.

Tasks:

- May take multiple timesteps to execute (timing constructs OK).
- Blocking and non-blocking assignments allowed.
- May call functions or tasks

The major changes in SystemVerilog are:

- Optional ANSI-C style formal declarations
- All SystemVerilog types supported (default is logic)
- Default argument direction is input
- **automatic** keyword implies dynamic memory allocation at call-time which means multiple simultaneous calls and/or recursion is now possible. Also enables use of *ref* argument types.
- **return** keyword in tasks, **return()** in functions
- Functions can be declared **void** type, meaning no value is returned.
- Arguments may be passed by value or by reference

5.1 Task Declaration

```
task [automatic] taskname ( arg_list );  
    declarative_statements  
    procedural_statements  
endtask
```

5.2 Function Declaration

```
function [automatic] returntype funcname ( arg_list );  
    declarative_statements  
    procedural_statements  
endfunction
```

Notes:

- In Verilog, return value is specified by assigning it to the *func_name*.
- In SystemVerilog it is recommended to use the `return (value)` statement.

5.3 Argument lists

```
direction argtype argname [= defaultval], ...
```

direction can be one of:

input	Default if no direction specified, argument is copied in at beginning
output	Argument value is copied out of the task at end
inout	Copied in at beginning, copied out at end
ref	Passed by reference (no copy). ref is not allowed in static subroutines, only automatic ones.

Notes:

- In Verilog, functions may return only a single value by assignment and only input argument directions are supported.
- In Verilog, tasks support input, output and inout argument directions.
- In Verilog, all arguments are passed by copying into the subroutine.
- In SystemVerilog, arguments passed by reference do not copy. Instead the reference is passed to the subroutine which accesses the argument data via the reference.

5.4 Passing arguments by name

```
funcname ( .argname(val), .argname(val), ... );
```

5.5 Static and Automatic variables

static	Data allocated on instantiation and never de-allocated
automatic	Stack storage allocated on entry to task, function, or named block and de-allocated on exit

Section 6: Processes

6.1 Procedural blocks

initial	Single-execution stream (same as Verilog)	(SV)
always	Repeated-execution stream (same as Verilog)	
always_ff	Explicitly models sequential logic	
always_comb	Explicitly models combinational logic	
always_latch	Explicitly models latched logic	

Notes:

- **always_ff**
 - Only one event control allowed
 - No blocking assignments allowed
 - Variables written by **always_ff** may not be written by any other process (Good Synthesis rule)
- **always_comb** and **always_latch**
 - Infer their own sensitivity list
 - Automatically triggered at time 0 after all other procedural blocks have triggered (reduce race conditions)
 - Variables written by either of these may not be written by any other process (Good Synthesis rule)

6.1.1 Procedural Timing Controls

Verilog supports three basic timing controls within procedural blocks

Timing Control	Description
#delay	Delay execution for specific time period. <i>delay</i> may be a literal value, an expression or a variable
@([edge] signal or [edge] signal or ...)	Delay to the next logical transition of a signal or any of a set of signals (valid delimiters are , and or). Optional <i>edge</i> may be posedge , negedge or not specified (any transition implied)
wait (expression)	Delay until expression evaluates to true. If expression is already true, no delay is implied.

Notes:

- **@()** timing control may be used with net or variable types (including vectors), named events or named procedural blocks.

6.1.2 Procedural Sensitivity lists

Specific to **always** block and its SystemVerilog derivatives, the sensitivity list is a list of signals (and optional transition edge), a change in any of which will trigger the **always** block. Tools (including Synthesis tools) can interpret from the sensitivity list whether sequential or combinational code is inferred.

Syntax	Inference
always @ (sig, sig, ...)	Combinatorial sensitivity list (no edges specified)
always @* - Verilog 2001 always_comb, always_latch - (SV)	Combinatorial logic inferred. Tools infer the sensitivity list from all signals used in procedure that follows. @* does not include signals found in called methods, SV syntax does
always @(posedge sig1 or negedge sig2) always @(posedge sig1 , negedge sig2)	Sequential sensitivity list Verilog 2001 alternative syntax

Notes:

- Verilog 2001 introduced @* as a shorthand to have the simulator or other tool infer the sensitivity list from an analysis of the procedural block in the same way that logic synthesis tools do.
- SystemVerilog supersedes @* with **always_comb** and **always_latch** blocks.
- The keyword **or** is used to declare a multi-signal sensitivity list, where a change in any signal will trigger the block.
- Verilog 2001 introduced the alternative syntax of a comma separated sensitivity list.

6.1.3 Sequential Block

Where more than a single statement is required to execute within a procedural block some grouping is required. A **begin ... end** block groups multiple statements together in such a way that they are evaluated in the order they are listed. See also *Dynamic Processes* for **fork ... join** groups.

6.1.4 Procedural Assignments

Syntax	Type
<code>[#delay] register_type = [# delay] expression;</code>	Blocking assignment
<code>[#delay] register_type <= [# delay] expression;</code>	Non-blocking assignment
<code>assign register_type = [#delay] expression;</code> <code>deassign register_type;</code>	Procedural Continuous assignment
<code>force net_or_register_type = [#delay] expression;</code> <code>release net_or_register_type;</code>	Procedural continuous assignment of any node.

Notes:

- Delay on LHS of an assignment implies a simple delay before that assignment statement is evaluated.
- Delay on RHS of an assignment implies a delay between the time the expression is evaluated and when it is actually assigned to the register_type on the left.
- Blocking assignments delay execution of subsequent statements in the same block until the blocking assignment has completed.
- Non-blocking assignments are scheduled via the event queue to execute at the end of the timestep indicated by the RHS timing control if present. They do not block subsequent statements in the same block.
- Procedural continuous assignments override any other procedural assignments to that register.
- **force/release** assignments override any other assignments or drivers of that node.

6.1.5 Blocking vs Non-blocking assignment: Rules of Thumb

Rule of Thumb 1:

For procedural blocks that describe combinational logic, use blocking assignments with NO delay

```
always @ (b or c) // Procedural model of 2 input and gate
    a = b & c;
```

Rule of Thumb 2:

For procedural blocks that describe sequential code, use non-blocking assignments exclusively and if desired, a small RHS delay to represent the clk-to-output delay of a flip-flop.

```
always @ (posedge clk) // Procedural model of D-type f/f
    q <= #1 d;
```

6.2 Dynamic Processes

A process is a thread of one or more programming statements operating independently of other threads of statements. The simplest type of process is static. Examples of static processes are **initial** and **always** blocks and continuous assignments.

SystemVerilog enhances and adds a second type of process called dynamic.

6.2.1 fork join

```
fork [: label]
    statements
join [: label]
```

Notes:

- Each statement is spawned into a concurrent process. The parent process continues past the join when all statements have completed.
- To combine more than one statement in a single concurrent process, use **begin ... end**.
- Because fork-join specifies a static number of process threads which are fully contained within their parent thread these are considered static processes in the SystemVerilog LRM.

6.2.2 fork join_any (SV)

```
fork [: label]
    statements
join_any [: label]
```

Notes:

- Each statement is spawned into a concurrent process. The parent process continues past the join when any one of the statements have completed.
- Can spawn truly independent threads so is considered a dynamic process.

6.2.3 fork join_none (SV)

```
fork [: label]
    statements
join_none [: label]
```

Notes:

- Each statement is spawned into a concurrent process. The parent process continues past the join immediately.
- Spawned processes do not execute until parent thread is suspended by a blocking statement (including the end of the parent process such as end of an **initial** block or of a **task**).
- Can spawn truly independent threads so is considered a dynamic process.

6.2.4 wait fork (SV)

wait fork

Notes:

- Waits for all immediate child processes of the parent procedural block (excluding their descendants) to complete

6.2.5 disable fork (SV)

disable fork

Notes:

- Terminates all active child processes of the calling process. This includes all descendant processes as well as child processes spawned from the root (e.g. initial/always) process.

6.2.6 Dynamic Process Methods (SV)

Part of default STD package. Allows fine-grained control of any process in the current simulation.

self()	Returns a handle to the current process.	
status()	Returns one of	
	FINISHED	The process terminated normally.
	RUNNING	The process is running (not in a blocking statement).
	WAITING	The process is waiting in a blocking statement.
	SUSPENDED	The process is stopped awaiting a resume.
	KILLED	The process was forcibly killed by a kill or disable.
kill()	Terminates a process and all of its subprocesses.	
await()	Allows a process to wait for the completion of another process.	
suspend()	Allows a process to suspend either its own execution or that of another process.	
resume()	Resumes a previously suspended process.	

Notes:

- **await()** is a task, all other methods are functions.

6.3 Inter-process Synchronization and Communication

- SystemVerilog provides several key features that help with resource sharing, synchronization and message passing between processes.

6.3.1 Named Events

event eventname

- As in Verilog, named events are an abstract type which has no direct h/w equivalent. SV offers several key enhancements however.

6.3.2 Triggering events

```
-> eventname  
->> eventname
```

Notes:

- The -> trigger operator unblocks all processes waiting on that event.
- The ->> trigger is a “nonblocking” event trigger, which triggers in the nonblocking assignment region of the scheduler.

6.3.3 Persistent trigger (SV)

```
eventname.triggered
```

Notes:

- This boolean property allows you to check the state of an event to see if it triggered in the current time step.

6.3.4 Event sequencing (SV)

```
wait_order( ev1, ev2 [, ev3, ... ] )  
statement1 [ else statement2 ]
```

Notes:

- wait_order suspends the calling process until all specified events occur in the order given. If the order fails, the statement(s) in the optional else clause are executed.

6.3.5 Semaphores (SV)

```
semaphore semname = new (num_keys);
```

Notes:

- A built-in class.
- A semaphore is a basic synchronization mechanism whereby a finite number of keys are defined and used to implement a fair resource sharing or mutex scheme. Think of it as a finite number of keys in a bucket, with processes attempting to grab one or more keys from the bucket.
- Semaphore is a built-in class that provides the following methods:
 - Create a semaphore with a specified number of keys: **new()**
 - Obtain one or more keys from the bucket: **get()**
 - Return one or more keys into the bucket: **put()**
 - Try to obtain one or more keys without blocking: **try_get()**

6.3.6 Semaphore Methods

<code>put(int N)</code>	Function, returns N keys back to the semaphore (default N=1).
<code>get(int N)</code>	Task, attempts to acquire N keys from the semaphore, blocks the process if the specified number of keys are not available. (default N=1).
<code>try_get(int N)</code>	Function, attempts to acquire N keys from the semaphore, returns 0 without blocking if the specified number of keys are not available. (default N=1).

6.3.7 Mailboxes (SV)

```
mailbox [#(non_singular_type)] mbname = new(size = 0);
```

Notes:

- A built-in class.
- A mailbox is a FIFO ordered communication mechanism that allows messages to be exchanged between processes. Data can be sent to a mailbox by one process and retrieved by another.
- Mailboxes are declared either as bounded or unbounded in size.
- Constructor argument size sets depth of Mailbox (default 0, unbounded)

6.3.8 Mailbox Methods

<code>num()</code>	Function, returns an int, the number of messages in the mailbox.
<code>put(msg)</code>	Task, stores message in the mailbox in FIFO order, blocks if the mailbox is full (bounded mailbox only).
<code>get(ref msg)</code>	Task, removes message from the mailbox in FIFO order, blocks if the mailbox is empty.
<code>peek(ref msg)</code>	Task, copies (without removing) the next message from the mailbox in FIFO order, blocks if the mailbox is empty.
<code>try_put(msg)</code> <code>try_get(ref msg)</code> <code>try_peek(ref msg)</code>	Non-blocking function forms of the above, return an int whose value is: ▪ 0 if mailbox is full/empty. ▪ Positive int if successful ▪ Negative int if type of index N is not same as mailbox

Section 7: Hierarchy

7.1 Modules

A module is the basic hierarchical structure in Verilog.

```
module name [ #( parameter list ) ][( port_list )];  
    port and type declarations  
    module or primitive instances  
    procedural blocks  
    continuous assignments  
    task and function declarations  
    specify blocks  
    assertion declarations  
endmodule
```

Notes:

- Modules may be instantiated one within another to form a hierarchy.

7.2 Module Port Declarations

The Verilog 2001 specification introduced ANSI-style port declarations to the language. This is a more readable and understandable style but the older Verilog 1995 style is still fully supported.

Verilog 1995 Syntax
<pre>module name [(port_list)]; port_direction [port_size] port_name ; ... endmodule</pre>
Example
<pre>module mux (a, b, y, sel) ; input [1:0] a, b; output[1:0] y; input sel ; ... endmodule</pre>

Notes:

- Ports are optional as are the parentheses after the module name.
- Port directions are: **input**, **output** or **inout** (or bidirectional).
- Port size may be scalar or vectored
- Vectored ports are declared with a range [msb:lsb] where both msb and lsb must be literal integers or parameters.

Verilog 2001 Syntax
<pre> module name [#(parameter_list)] [(ansi_style_port_list)]; ... endmodule </pre>
Example
<pre> module mux #(parameter width) (input wire [width-1:0] a, b, output reg [width-1:0] y, input reg sel); reg[width-1:0] internal_variable; ... endmodule </pre>

Notes:

- *parameter_list* is an optional list of parameters which may be used to determine the nature of internal structures as well as the ports of the module.
- *ansi_style_port_list* is a much improved syntax whereby all port information (direction, type, size and name) is specified within the parentheses of the module.

7.3 Module Instances

A module may be instantiated by port order or by port name. The latter is preferred, especially for modules with many ports.

<pre> //instantiation by port order (from module declaration) module_name instance_name (sig1, sig2, sig3, ...); //instantiation by port name (port declaration order irrelevant) module_name instance_name (.port3(sig3), .port1(sig1), .port2(sig2), ...); //instantiation of parameterized module (using by name syntax) module_name instance_name [#(.param_name(value), ...)] (.port3(sig3), .port1(sig1), .port2(sig2), ...); </pre>
--

Notes:

- Port order instantiation is prone to problems esp. if the module portlist should change. Name instantiation is preferred.
- Verilog 2001 made instantiation of parameterized modules easier by introduction of the third syntax shown above. Prior to this, `defparam` statements were used.
- **defparam** statement allows explicit parameter redefinition by hierarchical name: **defparam** hier_name.param_name = value;

7.3.1 Example - Module Instantiation

```
module mux ( a, b, y, sel );
input [1:0] a, b;
output[1:0] y;
input sel ;
. . .
endmodule

module test_mux;
reg[1:0] e, f;
reg s;
wire[1:0] g;
. . .
mux U1 ( e, f, g, s ); // Inst. by port order

mux U2 ( .y(g) .a(e), .sel(s), .b(f) );
           // Inst. by port name
. . .
endmodule;
```

Notes:

- This example illustrates both styles of module instantiation.

7.3.2 Example – Parameterized Module Instantiation

```
module mux #( parameter width = 8,
               parameter type mt = bit )
( input mt [width-1:0] a, b,
  output mt [width-1:0] y,
  input mt sel );
. . .
endmodule

module test_mux;
reg[15:0] e, f;
reg s;
reg[15:0] g;
. . .

mux U3 #( .width(16), .mt(reg) ) //Change type & size
( .y(g) .a(e), .sel(s), .b(f) );
. . .
endmodule;
```

Notes:

- Notice **parameter** type mt is used to change from **bit** to **reg** type mux.

7.3.3 Implicit Port Connections (SV)

Either of 2 shorthand methods can be used when module port declarations match signal names, type and size.

7.3.4 .name Syntax

```
modulename instname(.portname1, .portname2, ... );
```

7.3.5 .* Syntax

```
modulename instname( .* );
```

7.4 Primitives

Verilog supports a number of built-in primitives. These simple logic elements are typically implemented via lookup tables and so execute efficiently.

Built-in gates and switches	
n-input gates	and, nand, nor, or, xnor, xor
n output gates	buf, not
Tri-state gates	bufif0, bufif1, notif0, notif1
Pull gates	pulldown, pullup
MOS switches	cmos, nmos, pmos, rcmos, rnmos, rpmos
Bidirectional switches	rtran, rtranif0, rtranif1, tran, tranif0, tranif1

7.4.1 User defined Primitives (UDPs)

In order to augment the set of built-in primitives, Verilog supports an efficient table-driven syntax to easily define new primitive elements called UDPs. While the details of this syntax are beyond the scope of this document, an example is included below that should be pretty self-explanatory.

```
primitive multiplexer (mux, control, dataA, dataB);  
output mux;  
input control, dataA, dataB;  
  table  
    // control dataA dataB mux  
      0      1      ?    : 1 ; // ? = 0 1 x  
      0      0      ?    : 0 ;  
      1      ?      1    : 1 ;  
      1      ?      0    : 0 ;  
      x      0      0    : 0 ;  
      x      1      1    : 1 ;  
  endtable  
endprimitive
```

7.5 Specify Blocks

A block statement enclosed between keywords **specify** and **endspecify**, that describes the timing characteristics of paths across a module or primitive. Two types of timing characteristic can be described.

- Path delays (from input to output port across the module or primitive)
 - Simple path delays (e.g. from input to output of an or-gate)
 - Edge-dependant delays (e.g. from clk to Q of a D flip-flop)
 - State-dependant delays (e.g. latch delays)
- Timing checks on signal events occurring at module or primitive inputs.
Used to verify the timing performance of a design by making sure that critical events occur within given time limits. Available checks are:

Timing checks with a stability window	\$setup (data_event , reference_event , timing_check_limit [, [notifier]]);
	\$hold (reference_event , data_event , timing_check_limit [, [notifier]]);
	\$setuhold (reference_event , data_event , timing_check_limit , timing_check_limit [, [notifier] [, [stamptime_condition] [, [checktime_condition] [, [delayed_reference] [, [delayed_data]]]]]);
	\$recovery (reference_event , data_event , timing_check_limit [, [notifier]]);
	\$removal (reference_event , data_event , timing_check_limit [, [notifier]]);
	\$crecm (reference_event , data_event , timing_check_limit , timing_check_limit [, [notifier] [, [stamptime_condition] [, [checktime_condition] [, [delayed_reference] [, [delayed_data]]]]]);
Timing checks on clocks and control signals	\$skew (reference_event , data_event , timing_check_limit \$skew (reference_event , data_event , timing_check_limit [, [notifier]]);
	\$timeskew (reference_event , data_event , timing_check_limit [, [notifier] [, [event_based_flag] [, [remain_active_flag]]]);
	\$fullskew (reference_event , data_event , timing_check_limit , timing_check_limit [, [notifier] [, [event_based_flag] [, [remain_active_flag]]]);
	\$period (controlled_reference_event , timing_check_limit [, [notifier]]);
	\$width (controlled_reference_event , timing_check_limit , threshold [, [notifier]]);
	\$nochange (reference_event , data_event , start_edge_offset , end_edge_offset [, [notifier]]);

Notes:

- **notifier** is a variable, declared in the module where timing check tasks are invoked. Each time a timing violation occurs, the corresponding notifier is toggled.

7.6 Packages (SV)

```
package name;
  type declarations
  variable declarations
  task and function declarations
  assertion declarations
endpackage
```

Notes:

- Packages are a mechanism for sharing namespace between modules, interfaces and programs.
- Processes (**initial/always**) cannot be declared in a package. This includes nets with implicit continuous assignments.
- Hierarchical references are NOT allowed inside of a package.

7.7 Referencing Package Items

7.7.1 Fully qualified

```
packagename::itemname
```

7.7.2 Via importing

```
import packagename::itemname;  
import packagename::*;
```

Notes:

- The asterisk * is used like a wildcard to signify that ALL items in the package are imported.
- If an item identifier in the imported package conflicts with an identifier already in the importing package that item is (silently) not imported.

7.8 Exporting package items

A package may import from other packages. An export statement allows the importing package to *pass-along* some/all of the imported namespace.

7.8.1 Example of package export

```
package p1;  
int x, y;  
endpackage  
  
package p2;  
import p1::x;  
export p1::*; // exports p1::x as the name "x";  
// p1::x and p2::x are the same declaration  
endpackage
```

7.9 Time unit and precision (SV)

Time unit and precision declarations have the equivalent functionality of the `'timescale` compiler directives in Verilog but apply only to the scope of the module, program, interface, package or compilation unit in which they appear.

```
timeunit timeval;  
timeprecision timeval;
```

Notes:

- *timeval* is a time literal that must be a power of 10, followed without a space by a unit, which must be one of fs, ps, ns, us, ms, s, step.

7.10 Program Blocks (PB) (SV)

```
program prog_id ;  
    program_items  
endprogram
```

Notes:

- A **program** Block is similar to a module in that it can be instantiated in and connected to the design hierarchy.
- A PB is specifically used as an entry point for test-program execution.
- Assignments to elements declared within the PB have unique scheduling semantics in the reactive region set to reduce races between PB and design.
- Syntax allows for multiple PBs but more than one raises the possibility of race conditions between them.
- A PB may contain type and data declarations, subroutines and one or more procedural code streams. PB variables may not be referenced from outside that PB except (under certain circumstances) from another PB.
- A PB contains **initial** and **final** blocks:- as few as possible to minimize risk of race conditions between them. A PB cannot contain **always** blocks, or **module**, **primitive**, **interface**, or other **program** instances.
- If a PB drives design signals via a **clocking** block, another set of possible race conditions are eliminated.

7.11 Clocking Blocks (CB) (SV)

```
clocking blockname @( clock_event );  
[ default input #skew output #skew; ]  
input [ #skew ] signalname, ...;  
output [ #skew ] signalname, ...;  
endclocking
```

Notes:

- A **clocking** block assembles signals that are synchronous to a particular clock and makes their timing explicit. The **clocking** block supports writing testbenches at a higher level of abstraction.

7.11.1 Example clocking block

```
default clocking bus @(posedge clk);  
default input #1ns output #2ns;  
input ready, enable = top.mem1.enable;  
inout data;  
output negedge ack;  
input #1step addr;  
endclocking
```

Notes:

- This defines a CB named **bus**, clocked on **posedge clk**. All signals listed in the CB are synchronous to this clock and edge.
- **#delay** specifies a *skew*, which the simulator will apply when sampling or driving the signals via this CB. Default **input skew** is **#1step** and the default **output skew** is **0**.
- Typically, a signal is sampled at *skew* time units *before* the clock event. Similarly, output (or inout) signals are driven *skew* simulation time units *after* the corresponding clock event.
- A **#1step** input *skew* allows input signals to sample their steady-state values in the time step immediately before the clock event (i.e., the preceding Postponed region).
- Hierarchical expressions may be declared in a CB to simplify code (e.g. `enable = top.mem1.enable`)
- Signal **ack** is declared as synchronous to **negedge** of **clk**.
- One CB can be specified as the **default** for all cycle delay operations within a given module, interface, or program.

7.11.2 Example using a Clocking Block

```
program test( . . . );

// insert default clocking block bus above

initial begin
    ## 5;                                // delay of 5 default CB cycles
    if (!bus.ready && bus.data == 10 )
        // references via CB bus
        . . .
    else begin
        @(posedge bus.ready); // wait for posedge ready
        ##1 bus.data = 100; // data driven via CB bus
    end
end
endprogram
```

Notes:

- This code shows how signals may be sampled/driven synchronously via a CB by simply adding the suffix of *clocking_block_name*. (e.g. **bus.data**).
- The timing control **@(posedge bus.ready)** above, blocks the process until signal ready transitions from lo to hi as sampled by posedge clk.
- **##N** is handy shorthand for specifying synchronous delays relative to the default CB

7.12 Interfaces (SV)

An **interface** is a hierarchical block similar to a **module** or **program** block. Its purpose is to encapsulate the communication between blocks, enabling the communication to be modeled at any desired level of abstraction.

7.12.1 Interface Declaration

```
interface name [ ( port_list ) ] ;  
    signal declarations  
    modport declarations  
    clocking block declaration  
    task and function declarations  
    instances of other interfaces  
endinterface
```

7.12.2 Modport Declaration

```
modport portname ( clocking clocking_block_name,  
                   input signalnames,  
                   output signalnames,  
                   inout signalnames,  
                   import task taskname( args ),  
                   export task taskname( args ) );
```

Notes:

- A **modport** lists interface signals and their directions from the point of view of the user of the **modport**, not the interface.
- **modport** items define
 - the direction of data on signals (input, output, inout)
 - The direction of task and function calls (import, export)
- **import** tasks and functions are supplied by the interface and called by the user.
- **export** tasks and functions must be supplied by the user and are called by the interface.
- If a given task or function appears in both import and export lists the implementation does not exist within the interface at all and is effectively shared from user to user via the interface.
- In general it shall be an error for multiple users to export tasks or functions of the same name. However the **extern forkjoin** keywords can be used to permit this in the form of a broadcast mode.

7.13 Instantiating Interfaces

7.13.1 Example 1 (modport list in module header)

```
interface i2;
  logic a, b, c, d, e, f;
  modport master (input a, b, output c, d, e);
  modport slave (output a, b, input c, d, f);
endinterface

module m (i2.master i); // <=====
  ...
endmodule

module s (i2.slave i); // <=====
  ...
endmodule

module top;
  i2 itf;
  m u1(.i(itf));
  s u2(.i(itf));
endmodule
```

7.13.2 Example 2 (modport list in module instantiation)

```
interface i2;
  logic a, b, c, d, e, f;
  modport master (input a, b, output c, d, e);
  modport slave (output a, b, input c, d, f);
endinterface

module m (i2 i);
  ...
endmodule

module s (i2 i);
  ...
endmodule

module top;
  i2 itf;
  m u1(.i(itf.master)); // <=====
  s u2(.i(itf.slave)); // <=====
endmodule
```

Section 8: Generate, Config & Library Map Files

8.1 Generate Blocks

Generate Blocks are an efficient way to conditionally or multiply instantiate module items. They even allow for recursive module instantiation. There are two kinds of generate constructs: loops and conditionals.

Loop generates allow a single generate block to be instantiated into a model multiple times.

Conditional generates, (**if-generate** and **case-generate**) instantiate at most one generate block from a set of alternative generate blocks.

```
genvar gen_var_name, ... ;  
[ generate ]  
  genvar gen_var_name, ... ;  
  items_to_generate  
[ endgenerate ]
```

- Only permitted within modules.
- May not contain port declarations, parameter declarations, specify blocks, or specparam declarations.
- Evaluated during elaboration time.
- **genvar** is a positive integer used as a variable within a generate loop and to instantiate multiple copies of the generate block.
- **generate/endgenerate** blocks are optional but improve readability

Loop and Conditional generate structures:

```
for ( genvar_init ; genvar_expr ; genvar_iteration )  
  generate_block  
  
if ( constant_expression )  
  generate_block_or_null  
[ else generate_block_or_null ]  
  
case ( constant_expression )  
  genvar_value : generate_block_or_null  
  genvar_value : generate_block_or_null  
  ...  
  default [ : ] generate_block_or_null  
endcase
```

Where `generate_block` is:

```
begin: generate_block_name
    generate_item
    generate_item
    ...
end
```

- `generate_block_name` [optional] is used to give each generated item a unique instance name

Example of loop `generate`: Gray-code to binary converter

```
module gray2bin1 #( parameter SIZE = 8 ),
    ( output [SIZE-1:0] bin,
      input [SIZE-1:0] gray );

genvar i;
generate
    for (i=0; i<SIZE; i=i+1)
        begin: bit
            assign bin[i] = ^gray[SIZE-1:i];
            // i refers to the implicitly defined localparam whose
            // value in each instance of the generate block is
            // the value of the genvar when it was elaborated.
        end
    endgenerate
endmodule
```

Example of conditional `if-generate`: Paramaterized Multiplier

```
module multiplier(a,b,product);
    parameter a_width = 8, b_width = 8;
    localparam product_width = a_width+b_width;
    // cannot be modified directly with the defparam
    // statement or the module instance statement #
    input [a_width-1:0] a;
    input [b_width-1:0] b;
    output [product_width-1:0] product;

    generate
        if((a_width < 8) || (b_width < 8))
            begin: mult // instantiate a CLA multiplier
                CLA_multiplier #(a_width,b_width) ul(a, b, product);
            end
        else begin: mult // instantiate a Wallace-tree multiplier
            WALLACE_multiplier #(a_width,b_width) ul(a, b,
            product);
        end
    endgenerate
    // NOTE: The hierarchical instance name will be mult.ul
endmodule
```

Example of Conditional **case-generate**: DIMM Memory

```

module dimm(addr, ba, rasx, casx, csx, wex, cke, clk,
dqm, data, dev_id);
parameter [31:0] MEM_WIDTH = 16, MEM_SIZE = 8; // in mbytes
input [10:0] addr;
input ba, rasx, casx, csx, wex, cke, clk;
input [ 7:0] dqm;
inout [63:0] data;
input [ 4:0] dev_id;

genvar i;
case ({MEM_SIZE, MEM_WIDTH})
{32'd8, 32'd16}: // 8Meg x 16 bits wide
    begin: memory
        for (i=0; i<4; i=i+1) begin:word
            sms_08b216t0 p(.clk(clk), .csb(csx), .cke(cke),
                .ba(ba), .addr(addr), .rasb(rasx),
                .casb(casx), .web(wex),
                .udqm(dqm[2*i+1]), .ldqm(dqm[2*i]),
                .dqi(data[15+16*i:16*i]), .dev_id(dev_id));
            // The hierarchical instance names are memory.word[3].p,
            // memory.word[2].p, memory.word[1].p, memory.word[0].p,
            // and the task memory.read_mem
        end
        task read_mem;
            input [31:0] address;
            output [63:0] data;
            begin // call read_mem in sms module
                word[3].p.read_mem(address, data[63:48]);
                word[2].p.read_mem(address, data[47:32]);
                word[1].p.read_mem(address, data[31:16]);
                word[0].p.read_mem(address, data[15: 0]);
            end
        endtask
    end
{32'd16, 32'd8}: // 16Meg x 8 bits wide
    begin: memory
        for (i=0; i<8; i=i+1) begin:byte
            sms_16b208t0 p(.clk(clk), .csb(csx),
                .cke(cke), .ba(ba), .addr(addr),
                .rasb(rasx), .casb(casx), .web(wex), .dqm(dqm[i]),
                .dqi(data[7+8*i:8*i]), .dev_id(dev_id));
            // The hierarchical instance names are memory.byte[7].p,
            // memory.byte[6].p, ..., memory.byte[1].p, memory.byte[0].p,
            // and the task memory.read_mem
        end
    end

```

```

task read_mem;
input [31:0] address;
output [63:0] data;
    begin // call read_mem in sms module
        byte[7].p.read_mem(address, data[63:56]);
        byte[6].p.read_mem(address, data[55:48]);
        byte[5].p.read_mem(address, data[47:40]);
        byte[4].p.read_mem(address, data[39:32]);
        byte[3].p.read_mem(address, data[31:24]);
        byte[2].p.read_mem(address, data[23:16]);
        byte[1].p.read_mem(address, data[15: 8]);
        byte[0].p.read_mem(address, data[ 7: 0]);
    end
endtask
end
// Other memory cases ...
endcase
endmodule

```

8.2 Library Mapping Files

```

library identifier file_path [ { , file_path } ]
    [ -incdir file_path { , file_path } ] ;

include file_path ;

```

A logical collection of design elements, mapped to particular source files.

- **include** - like ``include`, inserts contents of a library map file in another file during parsing.

Notes:

- Introduced by Verilog 2001
- Logical group of design elements (modules, primitives, configs..)
- Tool specific implementation: might be source-code or precompiled library.
- Compiler must read these before ANY Verilog source is read
- If more than 1 defined, they will be treated in order read

8.3 Configurations

```
config config_identifier ;  
    design_statement  
    config_rule  
endconfig
```

An explicit set of rules to specify the exact source description to be used to represent each instance in a design.

- *design_statement* - design identifier
- *config_rule* - **default** | **liblist** | **instance** | **cell** | **use**

Notes:

- Introduced by Verilog 2001
- Supports both source-code or precompiled library modes.
- Steers elaboration step by identifying which library a particular instantiation will be drawn from.

8.4 Example of library & config

File: my_libs.map

```
library rtlLib ./addr.v ;    // this file compiles into rtlLib  
library gatLib ./mult.v , ./gat_adder/* ;
```

File: my_config.cfg

```
config my_config;  
    design work.top;           // top-level design module and its location  
  
// default source for sub-modules of top  
default liblist gatLib;  
  
// override default for specific instance top.il  
instance top.il liblist rtlLib;  
    cell mult use gatLib;     // redundant!  just a syntax example  
endconfig
```

Each simulator will have unique mechanisms to handle configurations / libraries. The following is an example of Mentor ModelSim/Questa use:

```
vlib work  
vlib gatLib  
vlib rtlLib
```

```
vlog my_libs.map my_config.cfg gat_adder/*.v mult.v addr.v top.v  
vsim -c my_config
```

Section 9: Classes

9.1 Class declaration

```
[ virtual ] class name [ extends base_class_name ] ;  
    property element declarations  
    method and constructor declarations  
endclass
```

Notes:

- A class is a datatype containing member properties (e.g. arrays, variables) and member methods (task or function declarations) which typically act as an API to manipulate the properties.
- A variable of the class type is called a handle and is used to reference the object in memory.
- Classes may be instantiated within program and interface blocks where module instantiations are strictly forbidden.
- Inheritance (**extends** keyword) allows a class to be written with another class (referred to as base or parent) as a starting point. Thus, it is possible to build derived classes that specify a base class then proceed to make changes or additions.

9.2 Class Constructors

```
function new ( args ) ;  
    statements  
endfunction
```

Notes:

- Every class is given a default constructor at compile time which allocates memory space and returns the starting address in memory where the object resides.
- Handles are **null** in value until they are assigned the return value of a call to the class constructor **new()**.
- Classes may require custom initialization in order to start executing. A custom constructor is created by declaring a function called **new()** within the class. This function has no return type, is totally non-blocking and shall be executed at creation of any instance of the class.

9.3 Object Instantiation

<code>classname objname;</code>	<i>Object is null</i>
<code>classname objname = new(args);</code>	<i>Object is initialized</i>

Notes:

- An instance of a class is called an object and is created by invoking the class constructor **new()**.
- Classes may be instantiated (an object created) at compile time but objects may also be created at run-time. This means objects may be created and destroyed at will during simulation.
- The keyword **this** denotes a predefined object handle referring to the object within which **this** is used.
- The keyword **super** is used from within a derived class to refer to inherited members of the parent (or base) class.

9.4 Constructor chaining

Notes:

- When a derived class is instantiated and its constructor (default or custom) is invoked, the first action taken by that constructor is to invoke the constructor of its base class and so on up to the topmost level. Thus all constructors in the class hierarchy are called in the correct order from root class to current object.
- If the constructor of a base class requires arguments, these may be provided (1) by default assignments in the base class or (2) in the declaration of the derived class or (3) by use of the **super** keyword.

```
class Packet;          // base class
    integer a;

    function new(int init = 5);          // method 1
        a = init;
    endfunction
endclass

class Ether_packet extends Packet(5); // method 2

    function new();
        . . .
    endfunction
endclass

class Ether_packet extends Packet(); // method 3

    function new();
        super.new(5);
    endfunction
endclass
```

9.5 Copying Objects

```
objB = objA;  
objB = new (objA);  
objB = new (); objB.copy(objA);
```

Notes:

- The first form does not do a copy at all. It creates a second reference to the original object.
- The second form does a “shallow” copy – a new object is created and all members are copied. However, any internal object members are referenced, not copied.
- The third form uses a custom method called **copy** within objB. The writing of a custom method like this is the only way to perform a “deep” copy, whereby all members are newly created and copied.

9.6 Controlling access to Members

```
local data_or_method_declaration;  
protected data_or_method_declaration;
```

Notes:

- By default, all members of a class are “public” and visible (accessible) from both inside and outside of the class.
- Members of a base class qualified as **local** are visible within the local scope and by hierarchical reference from other instances of the same base class. They are not visible from outside of the class. They are inherited but are not directly visible in the derived class. They *may* be accessed via inherited base class methods.
- Members of a base class qualified as **protected** are effectively the same as local members of that class except that when inherited, protected members are directly visible within the derived class.

9.7 Virtual Methods

```
virtual method_declaration;
```

Notes:

- A virtual method is like a prototype declaration. A class derived from the base class is considered likely to provide an implementation of the method. In a hierarchical inheritance chain the latest (lowest) implementation is the “only” one.
- Unless the class is itself declared virtual (see next section), virtual methods must be declared with an implementation.

9.8 Virtual (abstract) Classes

```
virtual class class_declaration;
```

Notes:

- A virtual class is a placeholder class which may not itself be instantiated.
- Virtual methods within a virtual class may be declared without a statement body. Overriding of such methods in derived classes is not compulsory.
- If a virtual method with no implementation were not overridden in a derived class and yet executed, it would do nothing as its structure implies.

9.8.1 Pure Virtual Methods

A pure virtual method is a member of a virtual class and qualified with the word *pure*. Not only do such methods have no implementation, they don't even have an `endfunction/endtask` keyword. Pure virtual methods **MUST** be overridden in a derived class.

```
virtual class BasePacket;  
pure virtual function integer send(bit[31:0] data);  
// No implementation OR endfunction  
endclass  
  
class EtherPacket extends BasePacket;  
virtual function integer send(bit[31:0] data);  
// body of the function  
...  
endfunction  
endclass
```

9.9 Defining Methods Outside Class Declaration

```
class classname;  
    extern function returntype funcname( args );  
endclass  
  
function returntype classname::funcname( args );  
    function body  
endfunction
```

9.10 Parameterized Classes

```
class name #( param_list )  
[ extends base_class_name ( arg_list ) ] ;  
  data element declarations  
  method and constructor declarations  
endclass
```

Notes:

- Constructors allow for customization of objects at **run** time.
- Parameters allow for customization of objects at **compile** time.
Parameters may be either value or type
- A parameterized class is called a generic class and does not create a new type. Each instance with unique parameters creates a new type called a specialization. An instance with the default parameters is called a default specialization.

```
class CC #( type T = int, int size = 512 );  
  T payload [size];  
endclass  
  
CC cc_handle = new(); // default specialization  
  // payload: type int, size [0:511 ]  
  
CC #(.T(integer), .size(1024)) cc_handle = new();  
  // payload: type integer, size [0:1023 ]
```

9.11 Virtual Interface

Notes:

- A mechanism for separating an abstract model (or object) from the actual low-level signals of the design.
- Think of a virtual interface as somewhat like a handle which points to a real interface. Remember, it is NOT a true handle which is an OOP concept.
- By defining a virtual interface within a class, different objects of that class can point to specific interfaces or the specified interface may be changed during the life of the object.
- Typically, virtual interface “handles” are initialized procedurally or via a dedicated class method. They may also be initialized via the constructor as shown in the following example.

```
class test_design();
    virtual intf_type #(6).abc v_intf;

    function new ( virtual intf_type #(6).abc real_if );
        // map virtual interface to real interface
        v_intf = real_if;
    endfunction
endclass

interface #(parameter DEPTH=5) intf_type;
    modport abc(input a, ref b, c);
    . . .
endinterface

module top;

    intf_type d_if(); // interface instance

    design DUT(.dutio ( d_if ));

    test_design t_DUT = new( d_if );

    . . . // rest not shown
endmodule
```


Section 10: Randomization & Constraints (sv)

SystemVerilog randomization capabilities are class-based. The member variables can be declared as randomized, and additional constraints can be declared inside a class.

```
class classname;
    rand type varname, varname, ...;
    randc type varname, varname, ...;
    constraint constraintname { constraint_block }
endclass
```

Random values are generated with a call to the `randomize()` method of an object `objectname.randomize()`

10.1 rand and randc

`rand` and `randc` can be applied to any integral data type or composite (array, structure).

<code>rand</code>	Standard random values uniformly distributed (may repeat).
<code>randc</code>	Random “cyclic” values cover all possible values before repeating.

10.2 Enabling and Disabling Randomization

Randomization can be enabled or disabled either through arguments to `randomize()` or with the `rand_mode()` method.

```
objectname.randomize (membervar1, membervar2, ...)
```

Notes:

- Any member variable listed will be randomized, whether or not it was declared with a `rand` directive. Any variable not listed will not be randomized, regardless of `rand(c)` directives.

```
object_or_member_name.rand_mode ( on_or_off )
```

Notes:

- `on_or_off` is a bit value. If 1, the object will be randomized as normal. If 0, randomization is disabled.

10.3 Constraint Block

Any legal SystemVerilog code can be used as constraint expressions with the following exceptions:

- Operators with side effects (++ , --) are not allowed.
- Only automatic functions with no output arguments or side effects can be used.

10.4 Constraint Block Constructs

10.4.1 foreach

```
foreach ( arrayname [ loopvar, loopvar,... ] ) expr ;
```

Notes:

- Applies an expression to each element of an array.
- Each loopvar corresponds to an array dimension. It is implicitly declared and its scope is the foreach construct.

10.4.2 implication (->, if else)

```
predicate -> expr;  
if (predicate) expr1 else expr2 ;
```

Notes:

- Predicate expressions are expressions that return a true or false (1 or 0) result.

10.4.3 dist

```
varname dist { weightspec, weightspec, ... };  
  
weightspec  
value := weight  
[lower:upper] := weight or [lower:upper] :/ weight
```

Notes:

- dist applies a weighted distribution to control random value generation (similar to labeled ping-pong balls drawn from a container).
- For ranges, the weight can also be applied with :/, which divides the weight equally across the range.

10.4.4 Solution order

The LRM requires that constraints are resolved simultaneously and that random values be uniformly distributed. This helps ensure equal exploration of the entire design space. It is possible however to force a solution order on variables.

```
solve identifier_list_a before identifier_list_b;
```

Notes:

- Does not affect solution space, cannot cause solver to fail
- Does affect probability densities

```
class AA;  
    rand bit a;  
    rand bit[1:0] b;  
    constraint ab { a -> b == 0; }  
    constraint sol { solve a before b; }  
endclass
```

Notes:

- Probabilities of variable values depend on their width:
 a (prob: 50%), b (prob: 25%)
- Constraint **ab** seems to imply that value of **a** determines value of **b** and hence **a** must be resolved first. Not true, in fact **a** and **b** are resolved simultaneously.
- Consider randomization of an **AA** object with and without constraint **sol**.

Without sol:

- **a** and **b** are resolved simultaneously.
- For **a == 0**, there are 4 possible values of **b**.
- For **a == 1**, there is only one value of **b** (zero).
- So, **a == 0** is 4 times more likely to occur than **a == 1**.

With sol:

- Solving **a** first gives it true 50% probability
- So, **b == 0** 50% of time, **b != 0** 50% of the time

10.5 In-line Constraints

Constraints can be declared at the point of calling the `randomize()` method.

```
objectname.randomize() with { constraint_block }
```

Notes:

- In-line constraints are applied in addition to constraints declared in the class.

10.6 Enabling and Disabling Constraints

```
objectname.constraint_mode( on_or_off );
```

Notes:

- on_or_off is a bit value. If 1, the object will be constrained as normal. If 0, constraints are disabled.

10.7 Randomizing Non-class Variables

```
std::randomize(var1,var2, ...) [with {constraint_block}];
```

Notes:

- Randomizes scope (non-class) variables. Optionally, in-line constraints can be added via the with clause.

10.8 Pre and Post randomization

```
object_name.pre_randomize();  
object_name.post_randomize();
```

Notes:

- These functions are automatically called by randomize() before and after number generation.
- While *pre_randomize()* and *post_randomize()* methods are not virtual, they are automatically called by the *randomize()* method, which is virtual. In this way they appear to behave as virtual methods..
- NOTE: These functions are NOT automatically chained as constructors are.

10.9 Manual Seeding

```
objectname.srandom( int seedval );  
process::self.srandom( string state );  
  
objectname.get_randstate();  
objectname.set_randstate( string state );
```

Notes:

- The *srandom()* method permits manual seeding of individual objects or threads. Use the process::self prefix from within a thread to seed it's RNG.
- *get_randstate()* returns the current state of the RNG of an object or thread in the form of a string. NOTE: strings are not portable across simulators.
- *set_randstate* sets the state of the RNG of an object.

10.10 Random Weighted Case

```
randcase
    weight_expr : statement;
    weight_expr : statement;
    ...
endcase
```

Notes:

- One branch is selected at random, weighted by the (integer) *weight* values. The probability of an individual branch being selected is (weight of branch) / (sum of all weights).

10.11 Random Sequence Generation

```
randsequence ( prod_id )
    production_id : production_id production_id ... ;
    production_id : production_id | production_id ;
    production_id : { statements } ;
    production_id : if (expr) prod_id else prod_id ;
    production_id : repeat (expr) production_id ;
    production_id : case (expr)
        expr : production_id ;
        expr : production_id ;
    endcase ;
endsequence
```

Notes:

- A *randsequence* block is used to randomly generate structured sequences of stimulus.

10.12 Random number system functions

```
function int unsigned $urandom [ (int seed ) ] ;
function int unsigned $urandom_range( int unsigned max,
                                     int unsigned min = 0 );
```

Notes:

- *\$urandom* returns a new unsigned 32-bit pseudo-random number.
- *\$urandom_range* returns an unsigned 32-bit integer within the specified range.

10.13 Random Distribution functions

```
$dist_normal( seed, mean, std_deviation )
$dist_exponential( seed, mean )
$dist_poisson( seed, mean )
$dist_chi_square( seed, degree_of_freedom )
$dist_t( seed, degree_of_freedom )
$dist_erlang( seed, k_stage, mean )
$dist_uniform( seed, start, end )
```

Section 11: Functional Coverage (sv)

11.1 Covergroup declaration

```
covergroup groupname (list_of_args) [clocking_event] ;  
    cover_option  
    cover_type_option  
    coverpoint_declaration  
    crosspoint_declaration  
endgroup
```

11.2 Coverpoint declaration

```
label : coverpoint expr [iff (expr)]  
{  
    bins name [[]] = { value_set } [iff (expr)];  
    bins name [[]] = (transitions) [iff (expr)];  
    bins name [[]] = default [sequence] [iff (expr)];  
    ignore_bins name = {value_and_transition_list};  
    illegal_bins name = {value_and_transition_list};  
}
```

Notes:

- Bins may be specifically declared but if they are not, auto bins will be implied.
- option.auto_bin_max sets the limit on number of auto bins per coverpoint.

11.2.1 Value bins

```
bins name = { val, val, [lwr:uppr], ... };  
wildcard bins name = (casex_expr);  
bins dyn_ary[] = {[lwr:upr]};  
bins fix_ary[N] = {[lwr:upr]};
```

Notes:

- If any user defined bins are declared, auto bins will NOT be created.

11.2.2 Transition bins

```
bins name = (val => val => val ...);  
bins name = (val,val,... => val,val,...);  
bins name = (val[*N], val[N:M], val[->N], val[=N]);
```

Notes:

- There is no such thing as an auto bins for transitions.

11.3 Crosspoint Coverage

```
label: cross coverpoint1, coverpoint2, ... [iff (expr)]  
{  
  bins name = binsof(binname) op binsof(binname) op ...;  
  bins name = binsof(binname) intersect([range]);  
}
```

Notes:

- The bins expression is a logical expression using operators !, &&, ||, intersect and grouping parentheses.

11.4 Cover Options

Name	Description	Default
weight = number	Statistical weight within level	1
goal = number	Target coverage	100
name = string	Text	
comment = string	Text	""
at_least = number	Minimum # of hits per bin	1
detect_overlap = boolean	Overlap between two bins	0
auto_bin_max = number	Maximum # of auto-bins	64
cross_auto_bin_max = number	Maximum # of auto cross-bins	
cross_num_print_missing = number	# of missing cross-product bins	0
per_instance = boolean	1 = Also track per covergroup instance	0

Notes:

- Options may be declared within a covergroup, coverpoint or cross.
- Options must be declared at the top of the covergroup, coverpoint or cross in order to have an effect on the content of that block.

11.5 Cover Type Options

<code>weight = constant_number</code>	Statistical weight within database
<code>goal = constant_number</code>	Target goal for covergroup
<code>comment = string_literal</code>	Text
<code>strobe = boolean</code>	Sample once per clk-event (end of timestep)

Notes:

- Type Options may be declared within a covergroup, coverpoint or cross and apply across these as a type.
- Where a setting is available in both Option and Type Option form, the Type Option is effectively a default value which may be overridden on an instance basis by a local Option.

11.6 Cover functions

<code>sample()</code>	Sample the covergroup
<code>get_coverage()</code>	Calculate coverage number (of all instances)
<code>get_inst_coverage()</code>	Calculate coverage number
<code>get_inst_name(string)</code>	Sets the instance name
<code>start()</code>	Start collecting coverage information
<code>stop()</code>	Stop collecting coverage information
<code>inst_query()</code>	Returns per-instance coverage info

11.7 Coverage System Functions

<code>\$set_coverage db_name()</code>	Name the coverage database file
<code>\$load_coverage_db(name)</code>	Load cumulative coverage info from file
<code>\$get_coverage()</code>	Returns overall coverage of all types

Section 12: SystemVerilog Assertions (sv)

12.1 Immediate Assertions

```
[label:] assert ( expr ) statements else statements;
```

Notes:

- Performs an immediate test of *expr* when the statement is executed inside a procedural block. If *expr* fails, the statements following the **else** are executed.

12.2 Severity System Tasks

<code>\$fatal (N, <i>message</i>)</code>	Run-time fatal
<code>\$error (<i>message</i>)</code>	Run-time error
<code>\$warning (<i>message</i>)</code>	Run-time warning, varies by tool
<code>\$info (<i>message</i>)</code>	No severity implied, just informative
<code>\$display (<i>message</i>)</code>	Like <code>\$info</code>

Notes:

- By default, an assertion that fails with no severity task called will trigger `$error()`. The message in the severity calls uses the same format as `$display()`.

12.3 Concurrent Assertions

```
[label:] assert_directive property( propname stmt1 )  
                                else stmt2 ;
```

Notes:

- Triggers checking of a property and specifies an optional action block to be executed when property matches (*stmt1*) or fails (*stmt2*).
- The `assert_directive` varies depending on the tool that is using the assertion and the intent of the user

12.4 Property Declarations

```
property name [ ( args ) ] ;  
    [clocking_event] [disable iff (expr)]  
    property_expression  
endproperty
```

Notes:

- *args* are optional and may be declared typed or untyped.
- **disable iff** is used to asynchronously abort (vacuous match) the checking of a property if and only if *expr* matches.

12.5 Assertion Directives

<code>assert property(pname)</code>	Evaluate property as verification
<code>cover property(pname)</code>	Evaluate property for coverage, capture statistics like number of attempts, passes, fails etc. only
<code>assume property(pname)</code>	Evaluate property for Formal Verification

Notes:

- By itself, a property does nothing. An assertion directive is needed to tell the simulator (or other tool) how to employ the property.

12.6 Expect Statement

expect (property_spec) <i>statement1</i> else <i>statement2</i> ;

Notes:

- An expect statement causes the executing process to block until the given property succeeds or fails.

```
integer data;
...
task automatic wait_for( integer value, output bit ok);
    expect( @(posedge clk) ##[1:10] data == value )
        ok = 1;
        else success = 0;
endtask

initial begin
    bit success;
    wait_for( 23, success ); // wait for the value 23
    ...
end
```

Notes:

- This task waits between 1 and 10 clock ticks for the variable *data* to equal a particular value, which is specified by the automatic argument *value*.
- The second argument, *ok*, is used to return the result of the *expect* statement: 1 for success and 0 for failure.

12.7 Property Expressions

```
sequence_expr  
antecedent_sequence_expr |-> consequent_sequence_expr  
antecedent_sequence_expr |=> consequent_sequence_expr  
combination of property expressions using and/or/not  
if ( expr_or_dist ) property_expr [else property_expr]
```

Notes:

- Implication operators allow a property to declare the exact antecedent sequence necessary to trigger checking of the property. If the antecedent sequence matches, the consequent sequence is attempted.
|> consequent attempt overlaps match of antecedent
|=> consequent attempt starts one sample tick after match of antecedent

12.8 Sequence Blocks

```
sequence name [ ( args ) ] ;  
    sequence_expression  
endsequence
```

Notes:

- Arguments are optional and may be declared typed or untyped.

12.9 Sequence Expressions

```
bool_expr seq_operator bool_expr ...  
first_match ( seq_expr, seq_match_item )
```

Notes:

- **first_match** allows detection of specifically the first match in a series of potential matches.

12.10 Sequence Endpoint

```
sequence_name.ended
```

Notes:

- The ended method cannot be used on a sequence with formal arguments.

12.11 Sequence Matchpoint

```
sequence_name.matched
```

Notes:

- The matched method allows detection of the endpoint of a subsequence which uses a different sampling clock from the current sequence.

12.12 Sequence Operators

[*M]	[*M:N]	Consecutive repetition
[=M]	[=M:N]	Non-consecutive repetition
[->M]	[->M:N]	Goto repetition (non-consecutive, exact)
##N	##[N:M]	Delay N cycles or any number from N to M cycles. N defaults to 1. M can be \$ for infinite range.
s1 throughout s2		s1 must hold during s2
s1 within s2		s1 must start at the same time or later than s2 and end at the same time or earlier than s2
s1 intersect s2		Both sequences must start at the same time, must hold, and end at same time
s1 and s2		Both sequences must start at the same time, must hold, may end at different times
s1 or s2		Either sequence must hold

Notes:

- The table above also shows operator precedence (highest at top).

12.13 Sequences in Procedural code

Sequences may be used in regular procedural code.

@(clocked_seq)	Block thread until <i>clocked_seq</i> matches
<i>clocked_seq</i>.triggered	True if <i>clocked_seq</i> has matched in current timestep, false otherwise

12.14 Data-use within a sequence or property

Sequences & properties may contain local variables which are automatic by definition.

```
sequence name [ ( args ) ] ;  
    [type automatic_var]  
    sequence_expression  
endsequence
```

Notes:

- All SV types are supported .
- Local variables may be sampled and manipulated over the life of the sequence or property.
- Hierarchical references to local variables are not allowed although they may be passed as arguments to sub-sequence calls.

(any_seq [, var_sample]) Variables may be sampled at the endpoint of a sub-sequence.

```
sequence aaa;  
byte x;  
    a ##1 ( b[->1], x = e )    ##1 ( c[*2], x = x + 1 );  
endsequence
```

Notes:

- Here byte x is a local automatic variable. Notice how it is sampled in one sub-sequence, then updated in a later sub-sequence.
- Methods are allowed so long as they are pure & automatic

12.15 Subroutine calls on match of a sequence

Tasks, task methods, void functions, void function methods, and system tasks can be called at the end of a successful match of a sequence. The subroutine calls, like local variable assignments, appear in the comma separated list that follows the sequence.

```
( any_seq [, subroutine_call [, subroutine_call] )
```

Notes:

- Subroutines must be pure & automatic
- For each successful match, the attached calls are executed in the order they appear in the list.
- The subroutines are scheduled in the Reactive region, like an action block.

12.16 Assertion System Functions

These functions are useful inside boolean expressions:

<code>\$onehot(expr)</code>	True if <i>only</i> one bit of <code>expr</code> is high
<code>\$onehot0(expr)</code>	True if <i>at most</i> one bit of <code>expr</code> is high
<code>\$isunknown(expr)</code>	True if any bit of <code>expr</code> is 'X'
<code>\$countones(expr)</code>	Returns number of 1's in a vector
<code>\$inset(expr, expr1[,exprN])</code>	True if <code>expr</code> matches one of <code>expr1-N</code>
<code>\$past(expr [,N][,expr1] [,clking_event])</code>	Return value of <code>expr</code> <code>N</code> ticks into the past. <code>expr1</code> is an optional qualifier on <code>clking_event</code> . <code>N</code> defaults to 1

```
sequence s_rd_3;
    ##3 (out_busData == mem[$past( busAddr,3 )]);
endsequence

property p_rd_3;
    @(posedge clk) read |-> s_rd_3;
endproperty
```

Notes:

- This example shows `$past` used to return value that was on `BusAddr` 3 ticks previous.

12.17 Value Change Functions

These functions describe transitions between 2 consecutive sample events:

<code>\$rose(expr [,clking_evt])</code>	True if LSB of <code>expr</code> went low-high
<code>\$fell(expr [,clking_evt])</code>	True if LSB of <code>expr</code> went high-low
<code>\$stable(expr [,clking_evt])</code>	True if no change between previous sample of <code>expr</code> and current sample
<code>\$sampled(expr [,clking_evt])</code>	Returns the sampled value of the expression with respect to the last occurrence of the clocking event. Allowed in assertions but redundant - as the sampled value of the expression is the same as the expression itself

Notes:

- Value change functions may be used outside of assertions in regular SV code.
- The sensitivity list of the procedural block where they are invoked specifies the sample clock.

12.18 Bind directive

The bind directive allows runtime instantiation of an assertion container within a target module or module instance.

```
bind target container instance ( sig_list )
```

Notes:

- *target* is either a module or module_instance.
- *container* is the name of the module/program/interface used as an assertion container.
- *instance* is the name used for *container* when it is instantiated within *target*.
- *sig_list* is a list of all *target* signals referenced by the assertions within *container*.

```
module cpu_chk (input clk, a,b,c,d);  
  property p1;  
    @ (posedge clk) a |-> ##1 (b != (c ^ d));  
  endproperty  
  cpu_p1: assert property (p1);  
endmodule  
  
bind cpu cpu_chk CHK1( clk, enable, d_out, d_in, val );
```

Notes:

- Here, *target* is a module, so **bind** instantiates *container* within every instance of module *target*. To select a single instance make *target* the instance name.
- Unlike this example, if *sig_list* uses the actual names of signals within *target*, the bind directive could use *.** notation to simplify port connections.

12.19 Assertion Control

These functions are useful to enable/disable individual or groups of assertions:

\$assertoff [levels [,list of modules or assertions)]]	Stop new checking, current checks complete unaffected
\$asserton (expr)	Start checking
\$assertkill (expr)	Like \$assertoff but aborts (vacuous match) any current attempts

Section 13: AVM Quick Reference

13.1 Named Component

instantiation methods

class avm_named_component extends avm_report_client

```
function new(string name, avm_named_component p=null,  
             bit check_parent=1)  
virtual function void remove()  
function avm_named_component absolute_lookup(string name)  
function avm_named_component relative_lookup(string name)  
virtual function void end_of_elaboration()  
virtual function void report()  
virtual function void configure()  
protected virtual function void export_connections()  
protected virtual function void import_connections()  
protected virtual function void connect()  
virtual function void do_display(int max_level = -1,  
int level = 0,  
bit display_connectors = 0)
```

The following functions are called by do_test and should not be called directly.

```
function void do_configure()  
virtual task do_run_all()  
virtual function void do_kill_all()  
function void do_report()
```

The following functions are called by the elaborator and should not be called directly.

```
function void do_set_env( avm_env e )  
function void do_export_connections()  
function void do_connect()  
function void do_import_connections()  
virtual function void check_connection_size()  
function void do_end_of_elaboration()
```

hierarchical reporting methods

```
function void set_report_severity_action_hier  
(severity s, action a)  
function void set_report_id_action_hier (string id, action a)  
function void set_report_severity_id_action_hier  
(severity s, string id, action a)  
function void set_report_severity_file_hier  
(severity s, FILE f)  
function void set_report_default_file_hier(FILE f)  
function void set_report_id_file_hier (string id, FILE f)  
function void set_report_severity_id_file_hier  
(severity s, string id, FILE f)  
function void set_report_verbosity_level_hier(int v)
```


13.2 Threaded Component

virtual class `avm_threaded_component` extends `avm_named_component`

```
function new( string name , avm_named_component parent )
task do_run_all()
function void do_kill_all()
virtual function void kill()
virtual task suspend()
virtual task resume()
pure virtual task run()
virtual function void report()
```

13.3 Environment

virtual class `avm_env` extends `avm_named_component`

```
function new(string name = "env")
virtual task do_test()
virtual function void connect()
virtual function void configure()
virtual task run()
virtual function void report()
virtual function void kill()
virtual task suspend()
virtual task resume()
```

The following functions are called by `do_test` and should not be called directly.

```
task do_run_all()
function void do_kill_all()
```

order of execution in `do_test`

1. `report_header()`
2. `elaborate()`
3. `do_configure()`
4. `do_run_all()`
5. `run()`
6. `do_kill_all()`
7. `do_report()`

order of execution in `elaborate`

1. `do_set_env(this)`
2. `do_export_connections()`
3. `do_connect()`
4. `do_import_connections()`
5. `check_connection_size()`
6. `do_end_of_elaboration()`

13.4 Transaction

```
virtual class avm_transaction
  pure virtual function avm_transaction clone()
  pure virtual function string convert2string()
```

Required functions not in base class which must be provided by the user.

```
function void copy(input T t)
function bit comp(input T a ,input T b)
```

```
class avm_class_pair #(type T1=int, type T2=T1)
  extends avm_transaction
```

```
class avm_built_in_pair #(type T1=int, type T2=T1)
  extends avm_transaction
```

Here is a typical implementation of a clone function.

```
function avm_transaction clone();
  T t = new();
  t.copy( this );
  return t;
endfunction
```

Typically, \$cast is used to convert a cloned object to the proper subtype.

```
T new_t;
$cast( new_t, t.clone() );
```

13.5 Policies

```
class avm_built_in_comp #(type T=int)
  static function bit comp(input T a, input T b)
```

```
class avm_built_in_converter #(type T int)
  static function string convert2string (input T t)
```

```
class avm_built_in_clone #(type T=int)
  static function T clone(input T from)
```

```
class avm_class_comp #(type T=int)
  static function bit comp(input T a,input T b)
```

```
class avm_class_converter #(type T=int)
  static function string convert2string(input T t)
```

```
class avm_class_clone #(type T=int)
  static function T clone(input T from)
```

13.6 Utilities

```
class avm_in_order_comparator
    #(type T=int,
      type comp=avm_built_in_comp #(T),
      type convert=avm_built_in_converter #(T),
      type pair_type= vm_built_in_pair #(T))
    extends avm_named_component
    analysis_if #(T) before_export, after_export
    analysis_port #(pair_type) pair_ap

class avm_algorithmic_comparator
    #(type BEFORE=int,
      type AFTER=int,
      type TRANSFORMER=int_transform)
    extends avm_named_component
    analysis_if #(AFTER) after_export

    function void write(input BEFORE b)

virtual class avm_subscriber #(type T=int)
    extends avm_named_component
    analysis_imp #(this_type, T) analysis_export

    function new(string name, avm_named_component p)
    pure virtual function void write(input T t)

class avm_random_stimulus #(type trans_type = avm_transaction)
    extends avm_named_component
    avm_blocking_put_port #(trans_type) blocking_put_port

    function new(string name, avm_named_component parent)

    virtual task generate_stimulus(trans_type t=null,
        input int max_count=0)

    virtual function void stop_stimulus_generation()

class global_analysis_ports #(type T=int)
    static analysis_port #(T) s_analysis_ports[string]
    static function analysis_port #(T)
        get_analysis_port(string name)
```

13.7 TLM Channels

```

class tlm_fifo #(type T=int) extends avm_named_component
  avm_blocking_put_imp #(T, this_type) blocking_put_export
  avm_nonblocking_put_imp #(T, this_type) nonblocking_put_export
  avm_put_imp #(T, this_type) put_export
  avm_blocking_get_imp #(T, this_type) blocking_get_export
  avm_nonblocking_get_imp #(T, this_type) nonblocking_get_export
  avm_get_imp #(T, this_type) get_export
  avm_blocking_peek_imp #(T, this_type) blocking_peek_export
  avm_nonblocking_peek_imp
    #(T, this_type) nonblocking_peek_export
  avm_peek_imp #(T, this_type) peek_export
  avm_blocking_get_peek_imp
    #(T, this_type) blocking_get_peek_export
  avm_nonblocking_get_peek_imp
    #(T, this_type) nonblocking_get_peek_export
  avm_get_peek_imp #(T, this_type) get_peek_export

  avm_analysis_port #(T) put_ap, get_ap

  function new(string name, avm_named_component parent=null,
    int size = 1)

  function int size()
  task put(input T t)
  task get(output T t)
  task peek(output T t)
  function bit try_put(input T t)
  function bit try_get(output T t)
  function bit try_peek(output T t)
  function bit can_put()
  function bit can_get()
  function bit can_peek()

class analysis_fifo #(type T=int) extends tlm_fifo #(T)
  function void write(input T t)
  analysis_imp #(analysis_fifo #(T), T) analysis_export

  function new(string name, avm_named_component parent = null)

class tlm_req_rsp_channel #(type REQ=int, type RSP=int)
  extends avm_named_component
  function new(string name, avm_named_component parent = null,
    int request_fifo_size=1, int response_fifo_size=1)

  avm_put_export #(REQ) put_request_export
  avm_blocking_put_export #(REQ) blocking_put_request_export
  avm_nonblocking_put_export #(REQ) nonblocking_put_request_export
  avm_get_export #(REQ) get_request_export
  avm_blocking_get_export #(REQ) blocking_get_request_export
  avm_nonblocking_get_export #(REQ) nonblocking_get_request_export
  avm_peek_export #(REQ) peek_request_export
  avm_blocking_peek_export #(REQ) blocking_peek_request_export
  avm_nonblocking_peek_export #(REQ) nonblocking_peek_request_export
  avm_get_peek_export #(REQ) get_peek_request_export
  avm_blocking_get_peek_export #(REQ) blocking_get_peek_request_export
  avm_nonblocking_get_peek_export #(REQ)
    nonblocking_get_peek_request_export

```

```

avm_analysis_port #(REQ) request_ap

avm_put_export #(RSP) put_response_export
avm_blocking_put_export #(RSP) blocking_put_response_export
avm_nonblocking_put_export #(RSP) nonblocking_put_response_export
avm_get_export #(RSP) get_response_export
avm_blocking_get_export #(RSP) blocking_get_response_export
avm_nonblocking_get_export #(RSP) nonblocking_get_response_export
avm_peek_export #(RSP) peek_response_export
avm_blocking_peek_export #(RSP) blocking_peek_response_export
avm_nonblocking_peek_export #(RSP) nonblocking_peek_response_export
avm_get_peek_export #(RSP) get_peek_response_export
avm_blocking_get_peek_export #(RSP) blocking_get_peek_response_export
avm_nonblocking_get_peek_export #(RSP)
    nonblocking_get_peek_response_export
avm_analysis_port #(RSP) response_ap

class tlm_transport_channel #(type REQ=int, type RSP=int)
    extends tlm_req_rsp_channel #(REQ, RSP)

    function new(string name, avm_named_component parent = null)

    task transport(input REQ request, output RSP response)

```

13.8 TLM Interfaces

blocking interfaces

```

virtual class tlm_blocking_put_if #(type T=int)
    pure virtual task put(input T t)
virtual class tlm_blocking_get_if #(type T=int)
    pure virtual task get(output T t)
virtual class tlm_blocking_peek_if #(type T=int)
    pure virtual task peek(output T t)
virtual class tlm_blocking_get_peek_if #(type T=int)
    pure virtual task get(output T t)
    pure virtual task peek(output T t)

```

nonblocking interfaces

```

virtual class tlm_nonblocking_put_if #(type T=int)
    pure virtual function bit try_put(input T t)
    pure virtual function bit can_put()
virtual class tlm_nonblocking_get_if #(type T=int)
    pure virtual function bit try_get(output T t)
    pure virtual function bit can_get()
virtual class tlm_nonblocking_peek_if #(type T=int)
    pure virtual function bit try_peek(output T t)
    pure virtual function bit can_peek()
virtual class tlm_nonblocking_get_peek_if #(type T=int)

```

combines tlm_nonblocking_get_if and tlm_nonblocking_peek_if

combined interfaces

virtual class tlm_put_if #(type T=int)

combines tlm_blocking_put_if and tlm_nonblocking_put_if

virtual class tlm_get_if #(type T=int)

combines tlm_blocking_get_if and tlm_nonblocking_get_if

virtual class tlm_peek_if #(type T=int)

combines tlm_blocking_peek_if and tlm_nonblocking_peek_if

virtual class tlm_get_peek_if #(type T=int)

combines tlm_get_if and tlm_peek_if

bidirectional interfaces

virtual class tlm_blocking_master_if #(type REQ=int, type RSP=int)

pure virtual task **put**(input REQ req)

pure virtual task **get**(output RSP rsp)

pure virtual task **peek**(output RSP rsp)

virtual class tlm_nonblocking_master_if #(type REQ=int, type RSP=int)

pure virtual function bit **try_put**(input REQ req)

pure virtual function bit **can_put**()

pure virtual function bit **try_get**(output RSP rsp)

pure virtual function bit **can_get**()

pure virtual function bit **try_peek**(output RSP rsp)

pure virtual function bit **can_peek**()

virtual class tlm_master_if #(type REQ=int, type RSP=int)

combines tlm_blocking_master_if and tlm_nonblocking_master_if

virtual class tlm_blocking_slave_if #(type REQ=int, type RSP=int)

pure virtual task **put**(input RSP rsp)

pure virtual task **get**(output REQ req)

pure virtual task **peek**(output REQ req)

virtual class tlm_nonblocking_slave_if #(type REQ=int, type RSP=int)

pure virtual function bit **try_put**(input RSP rsp)

pure virtual function bit **can_put**()

pure virtual function bit **try_get**(output REQ req)

pure virtual function bit **can_get**()

pure virtual function bit **try_peek**(output REQ req)

pure virtual function bit **can_peek**()

virtual class tlm_slave_if #(type REQ=int, type RSP=int)

combines tlm_blocking_slave_if and tlm_nonblocking_slave_if

virtual class tlm_transport_if #(type REQ=int, type RSP=int)

pure virtual task **transport**(input REQ request, output RSP response)

analysis interfaces

virtual class analysis_if #(type T=int)

pure virtual function void **write**(input T t)

13.9 Reporting

```
virtual class avm_report_client
    function new(string name="")
```

basic reporting

```
function void avm_report_message(input string id,
                                input string mess,
                                int verbosity_level=300)
function void avm_report_warning(input string id,
                                input string mess,
                                int verbosity_level=200)
function void avm_report_error(input string id,
                               input string mess,
                               int verbosity_level=100)
function void avm_report_fatal(input string id,
                               input string mess,
                               int verbosity_level=0)
function void report_summarize(FILE f=0)
virtual function void report_header(FILE f=0)
```

hooks

```
virtual function bit report_hook(input string id,
                                 input string mess,
                                 input verbosity)
virtual function bit report_message_hook(string id,
                                         string mess,
                                         int verbosity)
virtual function bit report_warning_hook(string id,
                                         string mess,
                                         int verbosity)
virtual function bit report_error_hook(string id,
                                       string mess,
                                       int verbosity)
virtual function bit report_fatal_hook(string id,
                                       string mess,
                                       int verbosity)
function void reset_report_handler()
```

set report properties

```
function void set_report_max_quit_count(int m)
function void set_report_verbosity_level(int verbosity_level)
function void set_report_severity_action(severity s, action a)
function void set_report_id_action string id, action a)
function void set_report_severity_id action
                (severity s, string id, action a)
function void set_report_default_file(FILE f)
function void set_report_severity_file (severity s, FILE f)
function void set_report_id_file (string id, FILE f)
function void set_report_severity_id_file
                (severity s, string id, FILE f)
function void dump_report_state()
```

see avm_named_component for functions that change reporting properties hierarchically

misc reporting functions

```
function avm_report_handler get_report_handler()
function void set_report_handler(avm_report_handler hndlr)
function string get_report_name()
function void set_report_name(string s)
```

```

virtual function void die()
class avm_reporter extends avm_report_client
function new(string name="reporter")
typedef enum {MESSAGE, WARNING, ERROR, FATAL} severity
typedef enum action{ NO_ACTION, DISPLAY, LOG, COUNT,
EXIT, CALL_HOOK } action

```

13.10 Ports and Exports

```

virtual class avm_port_base #(type IF=dummy_if) extends IF
function new(string name , avm_named_component parent ,
avm_port_type_e port_type,
int min_size=1, int max_size=1,
bit check_parent=1)
function void connect(this_type provider)
function void remove()
function void connect_to_if(input IF_if)
function IF lookup_indexed_if(int i=0)
function void debug_connected_to(int level=0,int max_level=-1)
function void debug_provided_to(int level=0,int max_level=-1)

```

ports

```

class avm_blocking_put_port #(type T=int)
extends avm_port_base #(tlm_blocking_put_if #(T))
class avm_nonblocking_put_port #(type T=int)
extends avm_port_base #(tlm_nonblocking_put_if #(T))
class avm_put_port #(type T=int)
extends avm_port_base #(tlm_put_if #(T))
class avm_blocking_get_port #(type T=int)
extends avm_port_base #(tlm_blocking_get_if #(T))
class avm_nonblocking_get_port #(type T=int)
extends avm_port_base #(tlm_nonblocking_get_if #(T))
class avm_get_port #(type T=int)
extends avm_port_base #(tlm_get_if #(T))
class avm_blocking_peek_port #(type T=int)
extends avm_port_base #(tlm_blocking_peek_if #(T))
class avm_nonblocking_peek_port #(type T=int)
extends avm_port_base #(tlm_nonblocking_peek_if #(T))
class avm_peek_port #(type T=int)
extends avm_port_base #(tlm_peek_if #(T))
class avm_blocking_get_peek_port #(type T=int)
extends avm_port_base #(tlm_blocking_get_peek_if #(T))
class avm_nonblocking_get_peek_port #(type T=int)
extends avm_port_base #(tlm_nonblocking_get_peek_if #(T))
class avm_get_peek_port #(type T=int)
extends avm_port_base #(tlm_get_peek_if #(T))
class avm_blocking_master_port #(type REQ=int, type RSP=int)
extends avm_port_base #(tlm_blocking_master_if #(REQ, RSP))
class avm_nonblocking_master_port #(type REQ=int, type RSP=int)
extends avm_port_base #(tlm_nonblocking_master_if #(REQ, RSP))
class avm_master_port #(type REQ=int, type RSP=int)
extends avm_port_base #(tlm_master_if #(REQ, RSP))
class avm_blocking_slave_port #(type REQ=int, type RSP=int)
extends avm_port_base #(tlm_blocking_slave_if #(REQ, RSP))
class avm_nonblocking_slave_port #(type REQ=int, type RSP=int)
extends avm_port_base #(tlm_nonblocking_slave_if #(REQ, RSP))
class avm_slave_port #(type REQ=int, type RSP=int)
extends avm_port_base #(tlm_slave_if #(REQ, RSP))
class avm_transport_port #(type REQ=int, type RSP=int)
extends avm_port_base #(tlm_transport_if #(REQ, RSP))

```


All ports have a constructor with the following prototype:

```
function new(string name, avm_named_component parent,
            int min_size=1, int max_size=1)
```

```
class avm_analysis_port #(type T=int)
    extends avm_port_base #(analysis_if #(T))
    function new(string name, avm_named_component parent)
    function void write(input T t)
    virtual function void connect(port_type provider)
```

exports

```
class avm_blocking_put_export #(type T=int)
    extends avm_port_base #(tlm_blocking_put_if #(T))
class avm_nonblocking_put_export #(type T=int)
    extends avm_port_base #(tlm_nonblocking_put_if #(T))
class avm_put_export #(type T=int)
    extends avm_port_base #(tlm_put_if #(T))
class avm_blocking_get_export #(type T=int)
    extends avm_port_base #(tlm_blocking_get_if #(T))
class avm_nonblocking_get_export #(type T=int)
    extends avm_port_base #(tlm_nonblocking_get_if #(T))
class avm_get_export #(type T=int)
    extends avm_port_base #(tlm_get_if #(T))
class avm_blocking_peek_export #(type T=int)
    extends avm_port_base #(tlm_blocking_peek_if #(T))
class avm_nonblocking_peek_export #(type T=int)
    extends avm_port_base #(tlm_nonblocking_peek_if #(T))
class avm_peek_export #(type T=int)
    extends avm_port_base #(tlm_peek_if #(T))
class avm_blocking_get_peek_export #(type T=int)
    extends avm_port_base #(tlm_blocking_get_peek_if #(T))
class avm_nonblocking_get_peek_export #(type T=int)
    extends avm_port_base #(tlm_nonblocking_get_peek_if #(T))
class avm_get_peek_export #(type T=int)
    extends avm_port_base #(tlm_get_peek_if #(T))
class avm_blocking_master_export #(type REQ=int, type RSP=int)
    extends avm_port_base #(tlm_blocking_master_if #(REQ, RSP))
class avm_nonblocking_master_export #(type REQ=int, type RSP=int)
    extends avm_port_base #(tlm_nonblocking_master_if #(REQ, RSP))
class avm_master_export #(type REQ=int, type RSP=int)
    extends avm_port_base #(tlm_master_if #(REQ, RSP))
class avm_blocking_slave_export #(type REQ=int, type RSP=int)
    extends avm_port_base #(tlm_blocking_slave_if #(REQ, RSP))
class avm_nonblocking_slave_export #(type REQ=int, type RSP=int)
    extends avm_port_base #(tlm_nonblocking_slave_if #(REQ, RSP))
class avm_slave_export #(type REQ=int, type RSP=int)
    extends avm_port_base #(tlm_slave_if #(REQ, RSP))
class avm_transport_export #(type REQ=int, type RSP=int)
    extends avm_port_base #(tlm_transport_if #(REQ, RSP))
```

All exports have a constructor with the following prototype:

```
function new(string name, avm_named_component parent,
            int min_size=1, int max_size=1)
class avm_analysis_export #(type T=int)
    extends avm_port_base #(analysis_if #(T))
    function new(string name, avm_named_component parent=null)
    function void write(input T t)
```

Section 14: SystemVerilog Keyword Index

□	5	edge	27
always	27	else	15
always_comb*	27	end	28
always_ff*	27	endcase	15
always_latch*	27	endclass*	49
and	37	endclocking*	41
assert*	66	endconfig	49
assign	24	endfunction	25
assume*	66	endgenerate	45
automatic	25	endgroup*	62
before*	59	endinterface	43
begin	28	endmodule	34
bind*	71	endpackage*	34
bins*	62	endprimitive	37
binsof*	63	endprogram*	40
bit*	3	endproperty*	65
break*	17	endspecify	38
buf	37	endsequence*	61
bufif0	37	endtable	37
bufif1	37	endtask	25
byte*	4	enum*	4
case	15	event	31
casex	15	expect*	66
casez	15	export*	39,43
cell		extends*	49
chandle*		extern*	54
class*	49	final*	17
clocking*	41	first_match*	67
cmos	37	for	17
config	49	force	28
const*	3	foreach*	58
constraint*	58	forever	16
context*		fork	27
continue*	17	forkjoin*	30
cover*	66	function	25
covergroup*	62	generate	45
coverpoint*	62	genvar	45
cross*	63	highz0	1
deassign	24	highz1	1
default	15	if	15
defparam	35	iff*	62
design		ifnone	
disable	31	ignore_bins*	62
dist*	58	illegal_bins*	62
do*	15	import*	39

*indicates a new keyword not present in IEEE-1364

□		priority*	15
include	21,48	program*	40
initial	27	property*	65
inout	26	protected*	53
input	26	pull0	1
inside*	14	pull1	1
instance		pulldown	1
int*	4	pullup	1
integer	4	pulsestyle_oneevent	
interface*	43	pulsestyle_ondetect	
intersect*	67	pure*	54
join	27	rand*	57
join_any*	30	randc*	57
join_none*	30	randcase*	61
large		randsequence*	61
lib		rcmos	37
list		real	1
library	48	realtime	18
local*	53	ref*	26
localparam	3	reg	4
logic*	4	release	28
longint*	4	repeat	16
macromodule		return*	17
matches*	16	rnmos	37
medium		rpmos	37
modport*	43	rtran	37
module	34	rtranif0	37
nand		rtranif1	37
negedge	28	scalared	
new*	8	sequence*	68
nmos	37	shortint*	4
nor	37	shortreal*	1
noshowcancelled		showcancelled	
not	37	signed	3
notif0	37	small	
notif1	37	solve*	59
null*	50	specify	38
or	28	specparam	3
output	26	static*	26
package*	34	string*	6
packed*	7	strong0	1
parameter	3	strong1	1
pmos	37	struct*	13
posedge	28	super*	51
primitive	37	supply0	1

supply1	1
table	37
tagged*	16
task	25
this*	51
throughout*	67
time	1
timeprecision*	40
timeunit*	40
tran	37
tranif0	37
tranif1	37
tri	1
tri0	1
tri1	1
triand	2
trior	2
trioreg	1
type*	
typedef*	5
union*	13
unique*	15
unsigned	3
use	
uwire	
var*	
vectored	
virtual*	53, 54, 56
void*	1
wait	27, 31
wait_order*	32
wand	1
weak0	1
weak1	1
while	16
wildcard*	62
wire	1
with*	60
within*	68
wor	1
xnor	37
xor	37